Compiling Rounding: Fast Bit-Accurate Simulation of Numerical Hardware

ANONYMOUS AUTHOR(S)

Engineers evaluate new numerical hardware, such as tensor cores, by running candidate designs on real workloads in bit-accurate software models. These models reproduce each design's results exactly, but are typically written by hand and emulate each operation with a number library, so they are slow to build and slow to run. An expert can speed up a model by rewriting it to use native operations, but must then show by hand that it still reproduces the design's results, and redo this whenever the design changes.

We automate this rewriting by compiling a design's specification into a fast, bit-accurate software model. We present FPy, an executable language for these specifications with *first-class rounding contexts*, which control how operations round, including data-dependent rounding. We introduce *semantics-preserving rewrites* over arithmetic and rounding that lower each rounding to native operations. Two static analyses check when each rewrite preserves semantics by bounding each value's digits: *format inference*, formulated as an abstract interpretation, and a novel *digit-bound analysis*. We prove the analyses and rewrites sound in Lean. The FPy compiler targets C++ for CPUs and Triton for GPUs. Its models run up to 14.77× faster on CPUs and 181× on GPUs than hand-written bit-accurate models. In a case study, we evaluate 113 designs on LLM inference and matrix-multiplication accuracy in **[T]** hours, where hand-written models would take an estimated **[N]** days.

1 Introduction

Machine learning has spurred new numerical hardware that trades accuracy for performance and energy efficiency. For example, tensor cores [57] provide matrix multiply-accumulate instructions, $D = AB + C$. They multiply inputs in low-precision *number formats*, such as bfloat16 (BF16) [41] and FP8 [37, 56, 64]. These instructions vary widely, across vendors and generations, in how they accumulate the products: in precision, order, and rounding [44, 80]. For instance, NVIDIA's instructions truncate each product to a fixed number of bits below the largest product's exponent. To pick among candidate designs, hardware engineers weigh not only power, performance, and area, but also how each behaves on real workloads: does training converge, and does inference preserve model accuracy? Before implementing a design, engineers answer these questions with a *bit-accurate* model of each candidate, a program that produces exactly the results the design specifies. Evaluating more candidates improves the chance of finding a good design, so these models must be fast to run and fast to build.

Historically, engineers have built each model by hand [69], translating the design's specification into a program. Since CPUs and GPUs lack many of the specification's number formats and rounding modes, the program typically emulates them with a *number library* [27, 29, 33, 58, 83], often built in-house. Such models are *slow to run*: every library operation decodes, aligns, rounds, and re-encodes its operands in software, which limits the size of the workloads engineers can test. Alternatively, an expert can use native datatypes ad hoc [80], but must then reason that the model still matches its specification. Either way, these models are *slow to build*: each change to the specification requires careful editing and retesting of the model. Together, these costs limit how many designs engineers can evaluate, and promising designs may go unexplored.

We cast translating a specification into a model as a compilation problem, mapping operations in formats the processor lacks onto operations it supports. Existing compilers, like GCC and Clang, are *datatype-first*: each type conflates how a number is stored with how arithmetic on it rounds, so they keep each operation in its declared type, ruling out optimizations that compute in a different one.[1] A compiler for models, however, need only preserve the design's *numerical behavior*,[2] so

[1] Without native half-precision (FP16) arithmetic, GCC computes it in single precision (FP32), but only as a special case [31].
[2] Our models do not preserve side effects, such as IEEE 754's exception flags [36].

it can be *rounding-first*: it may choose any datatypes and target operations that preserve this behavior [27, 35]. Prior work [71] makes this principled, checking when roundings can be dropped, inserted, or modified.

We automate this approach in a compiler, generating models that are fast to run, fast to build, and faithful to their specifications. We present FPy, an executable specification language that captures a design's numerical behavior with two principles. First, FPy has no numerical datatypes: every numerical value is a real number. Second, rounding is explicit: each arithmetic operation rounds under a *rounding context* [16, 30], and, unique to FPy, *rounding contexts are first-class values*, so a specification can describe data-dependent rounding, such as a tensor core's truncated accumulation.

Rounding-first compilation comes at a cost: each target operation has fixed input and output datatypes, so the compiler must bound the values each expression produces. Fortunately, rounding and certain arithmetic operations bound the significant digits of their results. We exploit this in two static analyses: *format inference* [71], formulated as an abstract interpretation [14], bounds each expression's values by a number format, and a novel constraint-based *digit-bound analysis* bounds those digits relative to program values, so it stays precise even under data-dependent rounding. We then introduce *rewrites over both arithmetic and rounding* that lower each rounding to native arithmetic, building on prior work [71]; the analyses check when each is *semantics-preserving*. We prove the analyses and rewrites sound in Lean 4 [22].

The FPy compiler has two backends, C++ for CPUs and Triton [76] for GPUs; we evaluate them on 62 specifications of tensor core instructions. The compiler handles 60 of them, with a median compile time of 6.8 seconds for C++ and 7.1 seconds for Triton. Compared to hand-written models [80], our generated C++ code is between 4.71× and 14.77× faster (mean 7.03×), and our generated Triton code is between 5.8× and 181× faster (mean 100×). In a case study, we perform a design-space exploration with 113 candidate designs, evaluating them on LLM inference and matrix-multiplication accuracy in **[T]** hours, where hand-written models would take an estimated **[N]** days.

Concretely, this paper makes the following contributions:

- An executable specification language that captures numerical behavior independent of datatypes, where *rounding contexts are first-class values* (Section 3).
- An abstract interpretation based on *format inference* [71] that bounds each expression's values by a number format (Section 4.2).
- A constraint-based *digit-bound analysis* that bounds each number's significant digits relative to program values, even under data-dependent rounding (Section 4.3).
- A set of *semantics-preserving rewrites* over both arithmetic and rounding, which lower roundings to native arithmetic (Section 5).

Section 2 first gives an overview of our approach. Subsequently, Section 6 describes our compiler, Section 7 evaluates it, and Section 8 presents a case study. Section 9 discusses related work; Section 10 summarizes our conclusions. Our Lean mechanization is included as supplementary material.

2 Overview

Consider the dot product $\sum_i x_i y_i + c$ computed by a tensor core's matrix multiply-accumulate instruction. For most low-precision inputs, NVIDIA tensor cores compute each product exactly, truncate the products and c, add the truncated terms exactly, and round the sum to the output format [44, 80].[3] If we view each number as an infinite string of binary digits, where the digit at position j has weight 2^j, truncation drops the digits more than k positions below $e_{\max}$, a position computed from the inputs. Such designs cannot be modeled directly with the IEEE 754 operations

[3] These instructions are not correctly rounded, as IEEE 754 requires for basic operations [36].

that CPUs provide, which round only to a few fixed formats and never at a data-dependent position. Engineers *explore* candidate designs with differing behavior, varying k, the rounding, or how many terms are added at once, *build* a bit-accurate model of each one, and *evaluate* it by running an application's workload, such as LLM inference, on the model (Figure 1).

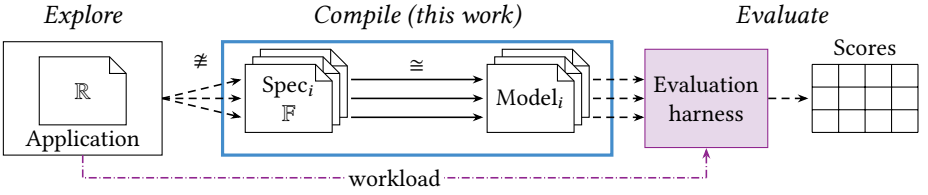


Fig. 1. Design-space exploration of numerical hardware. Candidate specifications over finite formats ($\mathbb{F}$) approximate ($\not\approx$) an exact definition ($\mathbb{R}$); instead of writing each model by hand, our compiler turns each specification into an equivalent model ($\cong$) to evaluate on the application's workload.

Building such a model by hand is slow: to model the dot product above, an engineer must compute $e_{\max}$, shift and truncate each product with bit operations, and add the truncated terms without losing bits. Changing k or how each term rounds means reworking this code, and any mistake changes the model's results, so every candidate's model must be tested. The model is also slow to run: it emulates the design's formats with a number library, so each arithmetic operation becomes a software routine rather than a single machine instruction.

Instead, our compiler generates the model from an FPy specification (*Compile* in Figure 1). An FPy specification has no numerical datatypes, and every rounding is explicit (Figure 2): each operation rounds under the *rounding context* set by its enclosing `with` statement or, outside one, by its caller. The specification in Figure 2 (left) follows the design step by step: `fdpa` computes the products exactly, finds $e_{\max}$, truncates the terms with `fused_sum`, and rounds the sum under `out`. Contexts are first-class values, so they can depend on data: `fused_sum` truncates at a position computed from the inputs, and `fdpa` takes its output context as an argument.

Ideally, the generated model would use the native arithmetic of a CPU or GPU rather than a number library's slower software routines. To do so, the compiler must store each value the specification computes in a datatype that represents it exactly, such as `float` or `int32_t`. With FP16 inputs, each exact product in `fdpa` has at most 22 significant digits, twice FP16's 11, so a `float`, with 24, holds it. Each truncated term in `fused_sum` has digits from position $e_{\max} - k$ to $e_{\max} + 1$, at most $k + 2$ significant digits, whatever the value of $e_{\max}$, so for $k = 25$ each term, scaled by $2^{k - e_{\max}}$, is an integer that fits in an `int32_t`.

Static analyses compute such bounds: format inference bounds the products by a format, and digit-bound analysis bounds the truncated terms relative to $e_{\max}$. Using these bounds, the compiler *rewrites the roundings themselves*, not just the arithmetic, until only native operations remain. The right side of Figure 2 shows the result for one design: each truncation is a scaling and an integer conversion, which drops the digits below position $e_{\max} - k$, and the final rounding is a conversion in the processor's round-toward-zero mode. The compiler thus chooses each datatype to fit the specification's rounding: it stores the products as `float`, the sum as `double`, and each truncated term as `int32_t`.

Our compiler addresses both problems. *Building a model is fast*: changing k or a rounding means recompiling, and since every rewrite is semantics-preserving, the model matches its specification. *Running the model is fast*: it computes entirely with native datatypes and operations.

```
def fused_sum(x, exp) =
  with Fixed(exp, RTZ) in
    t = [round(xi) for xi in x];
  with ℝ in
    ret sum(t)

def fdpa(x, y, c, out) =
  ps = join([xi * yi for xi, yi
             in zip(x, y)], [c]);
  emax = max_e(x, y, c);
  s = fused_sum(ps, emax - k);
  with out in
    ret convert(s)
```

```
double fused_sum(float x[17], float exp) {
  double down = std::ldexp(1.0, -exp);
  double up = std::ldexp(1.0, exp);
  double s = 0;
  for (int i = 0; i < 17; ++i) {
    int32_t t = (int32_t) (down * x[i]);
    s += up * t;
  }
  return s;
}

float fdpa(float x[16], float y[16], float c) {
  float ps[17];
  for (int i = 0; i < 16; ++i) ps[i] = x[i] * y[i];
  ps[16] = c;
  float emax = max_e(x, y, c);
  double s = fused_sum(ps, emax - 25);
  return convert(s);
}
```

```
        2^exp
          ↓
  1 0 1 1 |
      1 1 0 | 1
          1 | 1 1 1
          + | 1 0 0 1
  ----------|--------
  1 1 1 0 1 |
```

Fig. 2. An FPy specification of a tensor core's dot product (left) and its generated C++ (right, lightly edited) for FP16 inputs, $k = 25$, and `out` rounding toward zero to FP32, omitting special values and subnormals. Shaded statements set the context; Figure 4 gives `max_e` and `convert`. Bottom left: `fused_sum` drops the gray digits.

3 FPy: First-Class Rounding Contexts

Our approach starts with FPy, an *executable specification language*[4] with no numerical datatypes, in which every arithmetic operation rounds under a rounding context. To describe data-dependent rounding, *contexts are first-class values* (Section 3.1). We then use FPy to model matrix multiply-accumulate instructions (Section 3.2).

3.1 Syntax and Semantics

Figure 3 shows the syntax and semantics of FPy, which resembles a typical imperative language. FPy has no numerical datatypes: every number lies in $\mathbb{R}^* = \mathbb{R} \cup \{\pm\infty, \mathrm{NaN}\}$, the reals extended with infinities and NaN.[5] Lists are built with $[e_1, \ldots, e_m]$ and indexed with $e_1[e_2]$; records are built with $\{l_1 = e_1, \ldots, l_m = e_m\}$ and read with $e.l$.

Following FPCore [16, 30], a *rounding context* (or simply *context*) specifies a rounding function. During execution, FPy maintains a *current* context. A *context statement* (with e in s) sets the current context: it installs the context described by e for the duration of its body s. That context is either the *real context* ($\mathbb{R}$), whose rounding function is the identity, so operations under it do not round, or one built by a *context constructor* (ctx e) from a record.

In the big-step semantics, an expression e evaluates to a value v under a store σ and the current context C, written $\langle \sigma, C, e \rangle \Downarrow v$. A statement s yields an outcome o, written $\langle \sigma, C, s \rangle \Downarrow_S o$: either normal σ' with an updated store, or return v. A fixed map ρ gives the rounding function that each record describes, such as rounding to FP32 under round toward zero (RTZ). We write rnd_C for the rounding function of a context C: the identity for $\mathbb{R}$, and $\rho(v)$ for ctx v. A program Φ maps each function name to its parameter and body; functions may not be recursive. For brevity, the rules give each function one parameter; our examples use several. A program runs by calling an entry function under $\mathbb{R}$. Variables, lists, records, assignment, sequencing, conditionals, loops, assertions, and returns behave as usual.

[4] Since FPy is executable, we can differentially test compiled models against its interpreter [50].

[5] For simplicity, our semantics has no signed zero; our implementation distinguishes ±0 as IEEE 754 does.

$$
\begin{array}{rcll}
e & ::= & \mathsf{true} \mid \mathsf{false} \mid n \mid x \mid op(e_1, \ldots, e_k) \mid [\, e_1, \ldots, e_m \,] \mid e_1[e_2] & \text{expressions} \\
 & \mid & \{\, l_1 = e_1, \ldots, l_m = e_m \,\} \mid e.l \mid f(e) \mid \mathbb{R} \mid \mathsf{ctx}\ e & \\
s & ::= & x = e \mid s_1\,; s_2 \mid \mathsf{if}\ e\ \mathsf{then}\ s_1\ \mathsf{else}\ s_2 \mid \mathsf{while}\ e\ \mathsf{do}\ s & \text{statements} \\
 & \mid & \mathsf{ret}\ e \mid \mathsf{assert}\ e \mid \mathsf{skip} \mid \mathsf{with}\ e\ \mathsf{in}\ s & \\
op & ::= & + \mid - \mid \times \mid \div \mid \sqrt{\cdot} \mid \mathsf{abs} \mid \mathsf{logb} \mid \mathsf{round} \mid \ldots & \textit{Arith}\ \text{operators} \\
 & \mid & < \mid \leq \mid = \mid \mathsf{len} \mid \mathsf{isnan} \mid \ldots & \textit{Exact}\ \text{operators}
\end{array}
$$

$$
\begin{array}{rcll}
v & ::= & \mathsf{true} \mid \mathsf{false} \mid n \mid C \mid [\, v_1, \ldots, v_m \,] \mid \{\, l_1 = v_1, \ldots, l_m = v_m \,\} & \text{values} \\
C & ::= & \mathbb{R} \mid \mathsf{ctx}\ \{\, l_1 = v_1, \ldots, l_m = v_m \,\} & \text{rounding contexts} \\
o & ::= & \mathsf{normal}\ \sigma \mid \mathsf{return}\ v & \text{outcomes}
\end{array}
$$

$$
\textsc{E-Ctx}\quad \frac{\langle \sigma, C, e\rangle \Downarrow v \qquad v \in \mathrm{dom}(\rho)}{\langle \sigma, C, \mathsf{ctx}\ e\rangle \Downarrow \mathsf{ctx}\ v}
\qquad
\textsc{E-Arith}\quad \frac{op \in \mathit{Arith} \qquad \langle \sigma, C, e_i\rangle \Downarrow v_i \ \ (1 \le i \le k)}{\langle \sigma, C, op(e_1, \ldots, e_k)\rangle \Downarrow \mathrm{rnd}_C([\![op(v_1, \ldots, v_k)]\!])}
$$

$$
\textsc{E-Context}\quad \frac{\langle \sigma, \mathbb{R}, e\rangle \Downarrow C' \qquad \langle \sigma, C', s\rangle \Downarrow_S o}{\langle \sigma, C, \mathsf{with}\ e\ \mathsf{in}\ s\rangle \Downarrow_S o}
\qquad
\textsc{E-App}\quad \frac{\Phi(f) = (y, s) \qquad \langle \sigma, C, e\rangle \Downarrow v \qquad \langle [y \mapsto v], C, s\rangle \Downarrow_S \mathsf{return}\ v'}{\langle \sigma, C, f(e)\rangle \Downarrow v'}
$$

Fig. 3. Syntax and selected semantics of FPy, where x ranges over variables, f over functions, l over labels, and n over $\mathbb{R}^*$. Shaded forms are specific to FPy. The supplementary material lists all rules.

Rounding is *explicit*: operations are either rounded (*Arith*) or unrounded (*Exact*). Mathematical operations (arithmetic, algebraic, and transcendental functions) compute the exact result ($[\![\cdot]\!]$) and then round it under the current context C (E-Arith).[6] All other operations do not round: their results are not real-valued (e.g., comparisons) or are structural quantities (e.g., len).

Contexts are *first-class* values: programs can bind, pass, and return them (E-Ctx). The record of a context ctx v specifies its kind, such as IEEE 754 floating point or fixed point, and its parameters, such as precision, rounding mode, and overflow behavior. The fields of v can depend on program values, so rounding can be data-dependent. A context statement evaluates its expression under $\mathbb{R}$, so arithmetic on context parameters is exact (e.g., $p + 2$).

Contexts are *dynamically scoped*: a context statement sets the current context for its body (E-Context), and a called function inherits the current context (E-App). Since numbers have no datatypes, a function's arithmetic is not tied to a format, so functions are *context-polymorphic*: one definition runs under FP16, FP32, or FP64. Specifications are thus modular: a design composes reusable functions, such as `fma` in Figure 4, and each caller decides how the function rounds.

3.2 FPy by Example

Matrix multiply-accumulate instructions differ in how they compute the dot product of Section 2; we now show how the features of Section 3.1 model these differences, with code in Figure 4. We write contexts with shorthands: `IEEE754(es, nb, rm)` rounds to IEEE 754 floating point with `es` exponent bits, `nb` total bits, and rounding mode `rm`, such as *round to nearest, ties to even* (RNE); and `Fixed(n, rm)` rounds to fixed point with quantum 2^n. For brevity, our examples also use list comprehensions, `zip`, `join` (list concatenation), and `sum`, which adds a list's elements in order under the current context.[7]

Instructions first differ in how they compute the products. Some compute them exactly, as in Figure 2, where `fdpa` runs under $\mathbb{R}$; others round each product, as `cdna2` does with `ftz` (Figure 4),

[6] In the floating-point literature, E-Arith specifies *correct rounding*.

[7] We provide the definitions of these extensions in the supplementary material.

```
def fma(x, y, z) =
  with ℝ in
    t = x * y;
  ret t + z
```

```
def ftz(z) =
  with IEEE754(8, 32, RNE) in
    r = round(z);
  if abs(r) < 2^-126 then
    ret copysign(0, r)
  else
    ret r
```

```
def convert(s) =
  if s <= -2^128 or s >= 2^128 then
    with ℝ in
      ret copysign(inf(), s)
  else
    ret round(s)
```

```
def cdna2(x, y, c) =
  ps = [ftz(xi * yi) for xi, yi in zip(x, y)];
  s = ftz(ftz(ps[0] + ps[1]) + ftz(ps[2] + ps[3]));
  ret ftz(c + s)
```

```
def max_e(x, y, c) =
  es = [logb(xi) + logb(yi)
        for xi, yi in zip(x, y)];
  ret max(join(es, [logb(c)]))
```

Fig. 4. Excerpts from dot-product specifications; cdna2 models one step of an AMD CDNA2 instruction.

which rounds to FP32 and flushes tiny results to zero. Some also use *block scaling* [64], where a block of inputs shares one scale factor that multiplies their products exactly.

Most instructions, including the one in Figure 2, add the products in a single *fused sum*, which truncates every term at a shared position set by the largest exponent among the products and `c`; since contexts are first-class, `fused_sum` expresses this data-dependent rounding as `Fixed(exp, RTZ)`. The helper `max_e` computes this exponent, taking each product's exponent to be the sum of its factors', where `logb(x)` is $\lfloor \log_2 |x| \rfloor$. Others instead round after every addition, either in a tree, as in `cdna2`, or in a chain of FMAs: a loop computing `c = fma(xi, yi, c)` under `out`, where `fma` computes $x \times y$ exactly and rounds the sum under the caller's context (E-App).

Instructions also differ in when they add `c`: inside the fused sum, as in Figure 2, or after summing the products, as in `cdna2`. Finally, each instruction converts the result to its output format, as `fdpa` does under the context `out`. This conversion need not follow IEEE 754: `convert` returns infinity on overflow even under RTZ, where IEEE 754 returns the largest finite value.

Each choice takes only a few lines of FPy, so a specification can combine them; Section 8 evaluates many such combinations in a design-space exploration.

4 Analyzing FPy Programs

In FPy, numbers have no format and operations accept any number in $\mathbb{R}^*$, yet our compiler must store each number in a fixed datatype (Section 6) and map each operation to a target operation with fixed input and output datatypes (Section 5). We present two analyses that bound the numbers each expression can produce (Figure 5), both over abstract formats from prior work (Section 4.1). We formulate *format inference as an abstract interpretation* that bounds each expression by a format (Section 4.2), and present a *novel digit-bound analysis* that refines it where rounding depends on data (Section 4.3). We prove both analyses sound in Lean.

4.1 Background: Abstract Formats

Our analyses bound values by the *abstract number format* $\mathcal{A}$ of prior work [71], which parameterizes fixed- and floating-point formats. An instance $\mathcal{A}(p, exp, b, S)$ has a *maximum precision* $p \in \mathbb{Z}_{\geq 1} \cup \{\infty\}$, a *minimum quantum* 2^{exp} with $exp \in \mathbb{Z} \cup \{-\infty\}$, a *maximum representable value* b (possibly ∞), and a set $S \subseteq \{+\infty, -\infty, \mathrm{NaN}\}$ of *special values*. Its representable values are the numbers $m \times 2^e$ with integers $|m| < 2^p$ and $e \geq exp$ and magnitude at most b, together with the special values in S. Floating-point formats have finite p; fixed-point and integer formats have $p = \infty$. For example, FP32 is $\mathcal{A}(24, -149, (2^{24} - 1) \times 2^{104}, \{\pm\infty, \mathrm{NaN}\})$, 32-bit signed integers are $\mathcal{A}(\infty, 0, 2^{31} - 1, \emptyset)$,[8] and the unbounded fixed-point format with quantum 2^{exp} is $\mathcal{A}(\infty, exp, \infty, \emptyset)$.

[8] As in prior work, the bound is symmetric, omitting -2^{31}; our implementation bounds each sign separately.

```
1  def fused_sum(x, exp) =                 format inference                 digit-bound analysis
2    with Fixed(exp, RTZ) in
3      t = [round(xi) for xi in x];        ℝ* → 𝒜(24, −174, 2^128)          [emax − k, emax + 1]
4    with ℝ in
5      ret sum(t)                          ℝ* → 𝒜(32, −174, 2^133)          [emax − k, emax + 1 + ⌈log2 L⌉]
6
7  def fdpa(x, y, c, out) =
8    ps = [...];                           (FP16 ⊗ FP16) ⊔ FP32             [lsb(xi) + lsb(yi), msb(xi) + msb(yi) + 1] ⊔ digits(c)
9    emax = max_e(x, y, c);                {−149, −148, ..., 127}           val(emax)
10   s = fused_sum(ps, emax - k);          ℝ* → 𝒜(32, −174, 2^133)          [emax − k, emax + 1 + ⌈log2 L⌉]
11   with out in
12     ret convert(s)                      F_out
```

Fig. 5. Format inference (blue) and digit-bound analysis (green) on `fused_sum` and `fdpa` (Figure 2), for FP16 inputs, an FP32 accumulator, and k = 25, omitting special values. Red marks the trivial format $\mathbb{R}^*$; an arrow gives its refinement by digit bounds. Here, *msb* and *lsb* give the positions of a value's most and least significant digits, and $L = 17$ is the length of ps: 16 products and c.

Three properties are used throughout this section. First, *format containment* $F_1 \subseteq F_2$ has a simple, complete check on the formats' parameters. Second, format arithmetic bounds exact results: $x + y \in F_1 \oplus F_2$ and $x \times y \in F_1 \otimes F_2$ for $x \in F_1$ and $y \in F_2$, where $S_\oplus$ and $S_\otimes$ hold the special values a sum or product can produce (e.g., NaN for $\infty \times 0$). Third, a rounding context C *induces* a format F_C: $\mathrm{rnd}_C(x) \in F_C$ for every $x \in \mathbb{R}^*$, and $\mathrm{rnd}_C(v) = v$ for every $v \in F_C$, so F_C is the image of rnd_C.

4.2 Format Inference

Format inference [71] bounds the numbers each expression can produce by a format; we write $\mathrm{fmt}(e)$ for the format of e. Prior work defines it for straight-line programs, analyzing each assignment in execution order. FPy programs also have conditionals, loops, function calls, and contexts that are values, so we formulate format inference as an abstract interpretation [14] that mirrors the semantics of Figure 3 over abstract domains of formats and contexts. At each program point, it tracks a bound on each variable and the current context. At the start of a program, the analysis runs the entry function under $\mathbb{R}$, with each argument bounded by a format or context supplied with the program, such as FP16 for x and y in `fdpa` and FP32 for c. Figure 5 shows, in blue, the format it infers for each expression.

An abstract interpretation requires (i) an abstract domain with a partial order, (ii) a concretization γ mapping each abstract element to a set of concrete values, (iii) a join operation, and (iv) a sound abstract version of each operation. Format inference has two abstract domains: formats and abstract contexts. Formats are ordered by inclusion, and their concretization is the identity, $\gamma(F) = F$, since a format is itself a set of numbers. The join $F_1 \sqcup F_2$ is any format containing both, and an *abstract operator* $op^\sharp$ bounds the exact result of each operation. Abstract contexts form a flat lattice[9] of known contexts C, with $\gamma(C) = \{C\}$; the unknown context ? is the top element.

$$
\begin{array}{rcll}
F & ::= & \mathcal{A}(p, exp, b, S) \mid V \mid \mathbb{R}^* & \text{formats} \\
V & ::= & \{ n_1, \ldots, n_k \} & \text{finite sets}
\end{array}
\qquad
\begin{array}{l}
V_1 \sqcup V_2 = V_1 \cup V_2 \\
F \sqcup \mathbb{R}^* = \mathbb{R}^*
\end{array}
$$

$$
V \sqcup F = \begin{cases} F & \text{if } V \subseteq F \\ \lceil V \rceil \sqcup F & \text{otherwise} \end{cases}
\qquad
\begin{array}{l}
\mathcal{A}(p_1, exp_1, b_1, S_1) \sqcup \mathcal{A}(p_2, exp_2, b_2, S_2) = \\
\mathcal{A}(\max(p_1, p_2),\ \min(exp_1, exp_2),\ \max(b_1, b_2),\ S_1 \cup S_2)
\end{array}
$$

Fig. 6. Abstract domain of formats and its joins.

[9] That is, $\kappa ::= C \mid ?$, with $\kappa \sqcup \kappa = \kappa$ and $\kappa_1 \sqcup \kappa_2 = ?$ if $\kappa_1 \neq \kappa_2$, and $\gamma(?)$ is the set of all contexts.

An *abstract value* a is a format F, an abstract context κ, or a list or record of them. In Figure 6, a format is an instance of $\mathcal{A}$, a finite set V, which bounds numbers known exactly, such as constants and list lengths, or the top format $\mathbb{R}^*$, which bounds every number. Abstract operators act elementwise on finite sets, e.g., $V_1 +^\sharp V_2 = \{ n_1 + n_2 \mid n_1 \in V_1,\ n_2 \in V_2 \}$, and use the format arithmetic of Section 4.1 on instances of $\mathcal{A}$. Since sets can grow quickly, our implementation replaces a set above a fixed size limit by $\lceil V \rceil$, the smallest instance of $\mathcal{A}$ containing V.

We define format inference by structural recursion on the program; Figure 7 shows selected cases. Under an environment Γ mapping variables to abstract values and an abstract current context κ, $[\![e]\!]^\sharp_\kappa \Gamma$ bounds the value of an expression e by an abstract value, and $[\![s]\!]^\sharp_\kappa \Gamma = \langle \Gamma', a \rangle$ gives the environment Γ' after a statement s and a bound a on any value s returns.

Most cases are standard: they thread the environment through statements, bound all elements of a list by a single abstract value, and keep one abstract value per field of a record. A loop needs an invariant Γ^*, an environment that bounds both Γ and the environment after the body. The analysis finds one by iterating the body from Γ, widening any format still changing after a fixed number of iterations to $\mathbb{R}^*$, which ensures termination [14]. Two mechanisms are specific to FPy: rounding under an abstract context, and propagating it through the program.

Rounding under an abstract context. A rounded operation computes the exact result and then rounds it (E-Arith); format inference bounds both steps. The exact result is representable in $op^\sharp(F_1, \ldots, F_k)$, where $F_1, \ldots, F_k$ are the formats of the operands. For example, in line 8 of Figure 5, each product of two FP16 numbers is representable in FP16 $\otimes$ FP16. In line 12, κ is the known context `out`, so the result is representable in $F_{\texttt{out}}$. In line 3, however, κ is ?, since `Fixed(exp, RTZ)` depends on the argument `exp`, and only $\mathbb{R}^*$ bounds the result. The case for a rounded operation combines the two bounds with an *abstract rounding*: $\mathrm{rnd}^\sharp_C(F) = F$ if $F \subseteq F_C$, since rounding then has no effect, and F_C otherwise, while $\mathrm{rnd}^\sharp_?(F) = \mathbb{R}^*$, since ? may round to any format.

Propagating abstract contexts. A context statement `with` e `in` s analyzes s under the abstract context of e: a known context if every field of its record has one possible value, and ? otherwise. For example, `IEEE754(8, 32, RTZ)` is known, but `Fixed(exp, RTZ)` in line 2 of Figure 5 is not. A call analyzes the callee once per call site, under the caller's abstract context and the argument's abstract value; this terminates because FPy has no recursion. Contexts thus flow through arguments: if `fdpa` is called with a known context, `out` is constant in line 11.

Expression e	$[\![e]\!]^\sharp_\kappa \Gamma$
n	$\{n\}$
$op(e_1, \ldots, e_k)$	$\mathrm{rnd}^\sharp_\kappa(op^\sharp([\![e_1]\!]^\sharp_\kappa \Gamma, \ldots, [\![e_k]\!]^\sharp_\kappa \Gamma))$, where $op \in \mathit{Arith}$
$f(e_1)$	a, where $\Phi(f) = (y, s)$ and $\langle \Gamma', a \rangle = [\![s]\!]^\sharp_\kappa\, [y \mapsto [\![e_1]\!]^\sharp_\kappa \Gamma]$
Statement s	$[\![s]\!]^\sharp_\kappa \Gamma$
`with` e `in` s_1	$[\![s_1]\!]^\sharp_{\kappa'} \Gamma$, where $\kappa' = [\![e]\!]^\sharp_{\mathbb{R}} \Gamma$
`if` e `then` s_1 `else` s_2	$[\![s_1]\!]^\sharp_\kappa(\mathrm{C}^\sharp[\![e]\!]_\kappa \Gamma) \sqcup [\![s_2]\!]^\sharp_\kappa(\mathrm{C}^\sharp[\![\neg e]\!]_\kappa \Gamma)$
`while` e `do` s_1	$\langle \mathrm{C}^\sharp[\![\neg e]\!]_\kappa \Gamma^*,\ a \rangle$, where $\langle \Gamma', a \rangle = [\![s_1]\!]^\sharp_\kappa(\mathrm{C}^\sharp[\![e]\!]_\kappa \Gamma^*)$ and $\Gamma \sqcup \Gamma' \sqsubseteq \Gamma^*$

Fig. 7. Selected cases of format inference: $[\![e]\!]^\sharp_\kappa \Gamma$ is an abstract value, $[\![s]\!]^\sharp_\kappa \Gamma$ a pair $\langle \Gamma', a \rangle$ of the environment after s and a bound on what s returns, and $\mathrm{C}^\sharp[\![e]\!]_\kappa \Gamma$ the environment refined by the condition e; pairs join componentwise and environments pointwise. The supplementary material lists all cases.

Condition refinement. Hardware specifications use tests to handle special cases, such as overflow and subnormal inputs, and each test constrains the numbers that reach its branches. Format inference narrows each variable's format within a branch with a standard *condition test* [14]: $\mathrm{C}^\sharp[\![e]\!]_\kappa\,\Gamma$ shrinks each variable's format in Γ to the numbers that can satisfy e. Our implementation refines a variable x when a condition compares it, $|x|$, or $\mathrm{logb}(x)$ to a constant; under $\mathbb{R}$, the latter two are exact and unfold into comparisons on x. For example, $\mathrm{logb}(x) \geq k$ unfolds to $|x| \geq 2^k$; if $x \in \mathcal{A}(p, exp, b, S)$, it is a multiple of 2^{k-p+1}, so its format narrows to $\mathcal{A}(p, \max(exp, k-p+1), b, S)$.

Soundness. We lift γ to environments, $\gamma(\Gamma) = \{\, \sigma \mid \sigma(x) \in \gamma(\Gamma(x)) \text{ for every } x \in \mathrm{dom}(\Gamma) \,\}$, and to outcomes, $\gamma(\langle \Gamma', a\rangle) = \{\, \text{normal } \sigma' \mid \sigma' \in \gamma(\Gamma') \,\} \cup \{\, \text{return } v \mid v \in \gamma(a) \,\}$. Format inference is sound provided that each $op^\sharp$ bounds the exact result of op, each F_C the analysis computes is the format C induces, and condition tests are sound:

Theorem 4.1 (Soundness of format inference). *Suppose $\sigma \in \gamma(\Gamma)$ and $C \in \gamma(\kappa)$. If $\langle \sigma, C, e\rangle \Downarrow v$, then $v \in \gamma([\![e]\!]^\sharp_\kappa\,\Gamma)$; if $\langle \sigma, C, s\rangle \Downarrow_S o$, then $o \in \gamma([\![s]\!]^\sharp_\kappa\,\Gamma)$.*

The proof is by mutual induction on the evaluation derivations. In the central case, a rounded operation, the exact result lies in $op^\sharp(F_1, \ldots, F_k)$ by induction, and rounding under a known C either leaves it unchanged or yields a number in F_C, the image of rnd_C.

4.3 Digit-Bound Analysis

Format inference loses precision when a context depends on data: in `fused_sum`, the truncation position `emax - k` depends on the inputs, so each truncated term is bounded only by $\mathbb{R}^*$. The truncation does bound each term's digits, but only relative to `emax`, while a format bounds digit positions absolutely. *Digit-bound analysis* therefore relates digit positions to program values such as `emax` through constraints and solves them with Z3 [21] to bound each expression by a format.

Every finite number in a format has a finite binary expansion $v = \pm \sum_i d_i 2^i$ with digits $d_i \in \{0, 1\}$; the digit d_i has position i, e.g., $1.25 = 1.01_2$ has nonzero digits at positions 0 and -2. We write $D(v) = \{\, i \mid d_i = 1 \,\}$ for the positions of the nonzero digits of v, treating an infinity as one digit at position $+\infty$, so $D(\pm\infty) = \{+\infty\}$, and $D(F) = \bigcup_{v \in F} D(v)$ for a format F.

For each real-valued expression e with value v, the analysis introduces variables $lsb(e)$ and $msb(e)$ for $\inf D(v)$ and $\sup D(v)$, which are $+\infty$ and $-\infty$ for zero, since $D(0) = \emptyset$. We write $digits(e)$ for the interval $[lsb(e), msb(e)]$ and $digits(F)$ for $[\inf D(F), \sup D(F)]$. A constraint $digits(e) \subseteq [lo, hi]$ stands for $lo \leq lsb(e)$ and $msb(e) \leq hi$. Some integer-valued[10] expressions, such as `emax`, also get a variable $val(e)$ for their value. The constraints are linear, extended with max and min (Figure 8).

α	::=	$msb(e) \mid lsb(e) \mid val(e)$	variables
τ	::=	$n \mid \alpha \mid \tau_1 + \tau_2 \mid c \cdot \tau \mid \max(\tau_1, \tau_2) \mid \min(\tau_1, \tau_2)$	terms
ψ	::=	$\tau_1 \leq \tau_2 \mid \tau_1 = \tau_2$	constraints

Fig. 8. Constraint language of digit-bound analysis, where $n \in \mathbb{Z} \cup \{\pm\infty\}$ and $c \in \mathbb{Z}$.

The analysis generates a set Ψ of such constraints. An assignment ι of variables to $\mathbb{Z} \cup \{\pm\infty\}$ satisfies Ψ, written $\iota \models \Psi$, if it satisfies every constraint in Ψ. We write $\Psi \models \psi$ if every assignment that satisfies Ψ also satisfies ψ. A finite number v has precision $\sup D(v) - \inf D(v) + 1$, is a multiple of $2^{\inf D(v)}$, and has magnitude at most $2^{\sup D(v)+1} - 2^{\inf D(v)}$. Digit-bound analysis therefore bounds these quantities over the assignments $\iota \models \Psi$ where $\iota(msb(e))$ is finite: $prec_{\mathrm{D}}(e)$ is the largest $\iota(msb(e)) - \iota(lsb(e)) + 1$, $lsb_{\mathrm{D}}(e)$ the least $\iota(lsb(e))$, and $msb_{\mathrm{D}}(e)$ the greatest $\iota(msb(e))$. Each finite

[10] Integer constants, logb, expressions with inferred integer formats, and ±, max, and min of such expressions.

value of e then lies in the format $\mathrm{fmt}_{\mathsf{D}}(e) = \mathcal{A}(prec_{\mathsf{D}}(e), lsb_{\mathsf{D}}(e), b, \emptyset)$, where b is the largest number with at most $prec_{\mathsf{D}}(e)$ digits, all in $[lsb_{\mathsf{D}}(e), msb_{\mathsf{D}}(e)]$.

Figure 9 lists the constraints that selected expressions generate. The figure's Context column is the abstract context from format inference. Every expression e also gets $digits(e) \subseteq digits(\mathrm{fmt}(e))$ from format inference; the FP16 inputs of `fdpa`, for example, have no digit below position −24. After an assignment $x = e$, uses of x refer to $val(e)$ and $digits(e)$. At loops, the analysis is conservative: each variable x mutated in a loop gets fresh variables, constrained only by $digits(x) \subseteq digits(\mathrm{fmt}(x))$, so it analyzes the body once. We explain these constraints on `fused_sum` (Figure 5).

$$[l_1, m_1] +^{\mathsf{D}} [l_2, m_2] = [\min(l_1, l_2),\ \max(m_1, m_2) + 1] \qquad \mathrm{rnd}^{\mathsf{D}}_{\mathtt{Fixed}(\tau,\mathtt{RTZ})}([l, m]) = [\max(l, \tau), m]$$
$$[l_1, m_1] \times^{\mathsf{D}} [l_2, m_2] = [l_1 + l_2,\ m_1 + m_2 + 1] \qquad \mathrm{rnd}^{\mathsf{D}}_{\mathtt{Fixed}(\tau,rm)}([l, m]) = [\max(l, \tau), \max(m + 1, \tau)]$$
$$\mathrm{rnd}^{\mathsf{D}}_{\mathbb{R}}([l, m]) = [l, m] \qquad \mathrm{rnd}^{\mathsf{D}}_{\mathtt{IEEE754}(\ldots,\mathtt{RTZ})}([l, m]) = [\max(l, m - p + 1), m]$$

Expression e	Context	$val(e)$	$digits(e)$
$e_1 + e_2$	$\mathbb{R}$	$= val(e_1) + val(e_2)$	$\subseteq digits(e_1) +^{\mathsf{D}} digits(e_2)$
$\mathrm{logb}(e_1)$	$\mathbb{R}$	$= msb(e_1)$	$\subseteq [0, \infty]$
$\max(e_1, \ldots, e_k)$	$\mathbb{R}$	$\geq val(e_i)$	$\subseteq [\min_i lsb(e_i), \max_i msb(e_i)]$
$\mathrm{sum}(e_1)$	$\mathbb{R}$		$\subseteq [lo, hi + \lceil \log_2 L \rceil]$ if $\Psi \models digits(e_1) \subseteq [lo, hi]$
$op(e_1, \ldots, e_k)$	κ		$\subseteq \mathrm{rnd}^{\mathsf{D}}_{\kappa}(op^{\mathsf{D}}(digits(e_1), \ldots, digits(e_k)))$
$[e_1, \ldots, e_m]$	κ		$\subseteq [\min_i lsb(e_i), \max_i msb(e_i)]$

Fig. 9. Constraints that digit-bound analysis generates for selected expressions; L is the length of e_1, Ψ the constraints generated so far, and ^{D} marks the operators on digit intervals defined above, with p the precision of an IEEE 754 format. The supplementary material lists all cases.

Arithmetic and rounding. An arithmetic operation bounds the digits of its exact result with op^{D} and rounds that bound with $\mathrm{rnd}^{\mathsf{D}}_{\kappa}$: addition can carry into the next position, multiplication adds positions, and rounding under `Fixed` drops every digit below its position; except under RTZ, it may also carry into the next one. In line 8 of Figure 5, each product thus has $msb(\mathtt{ps}_i) \leq msb(\mathtt{x}_i) + msb(\mathtt{y}_i) + 1$, and each truncated term in line 3 has $msb(\mathtt{t}_i) \leq msb(\mathtt{ps}_i)$ and no digit below the truncation position. Summing L values with digits in $[lo, hi]$ gives digits in $[lo, hi + \lceil \log_2 L \rceil]$. For a call to `sum` on a list of known length, digit-bound analysis applies this summary instead of analyzing the loop in its body, which would weaken the running sum to its format.[11]

Relating values and digits. Digit positions and program values depend on each other: a `Fixed` context rounds at a position the program computes, and logb turns a position into a value. Value variables capture both directions: integer addition and subtraction combine values, the value of $\mathrm{logb}(e)$ is $msb(e)$, and max is at least each of its arguments. In `fused_sum`, the position is thus $val(\mathtt{emax}) - \mathtt{k}$, so $lsb(\mathtt{t}_i) \geq val(\mathtt{emax}) - \mathtt{k}$; and since `max_e` computes `emax` as the largest of $\mathrm{logb}(\mathtt{c})$ and the sums $\mathrm{logb}(\mathtt{x}_i) + \mathrm{logb}(\mathtt{y}_i)$, $msb(\mathtt{t}_i) \leq val(\mathtt{emax}) + 1$. Each truncated term therefore has at most $\mathtt{k} + 2$ significant digits for any $val(\mathtt{emax})$. By the summation rule, the sum in line 5 has digits in $[val(\mathtt{emax}) - \mathtt{k},\ val(\mathtt{emax}) + 1 + \lceil \log_2 L \rceil]$, and at most $\mathtt{k} + 2 + \lceil \log_2 L \rceil$ significant digits.

Soundness. Digit-bound analysis is sound if the format it infers for an expression contains every finite value of that expression. This relies on each op^{D} bounding the digits of op's exact result and each $\mathrm{rnd}^{\mathsf{D}}_{\kappa}$ bounding the digits of rounding under κ.

[11] Format inference has an analogous summary; see the supplementary material.

Theorem 4.2 (Soundness of digit-bound analysis). *Let e be an expression with finite $prec_{\mathrm{D}}(e)$, $lsb_{\mathrm{D}}(e)$, and $msb_{\mathrm{D}}(e)$. Whenever e evaluates to a finite number v, $v \in \mathrm{fmt}_{\mathrm{D}}(e)$.*

The proof first shows that some solution ι of Ψ has $\iota(lsb(e)) = \inf D(v)$ and $\iota(msb(e)) = \sup D(v)$, by induction on evaluation, building ι from the execution, where each variable takes its value; each constraint holds by a property of its operation, and those from format inference by Theorem 4.1.

Solving constraints. Computing $prec_{\mathrm{D}}(e)$, $lsb_{\mathrm{D}}(e)$, and $msb_{\mathrm{D}}(e)$ means solving three integer linear programs extended with max and min. In a second pass of format inference, the solver's bounds refine each expression's format: e also lies in $\mathrm{fmt}_{\mathrm{D}}(e)$, with the special values of its format added. If an optimum is unbounded or the solver times out, e keeps its format. These are the arrows in Figure 5. With Hopper's parameters ($\mathsf{k} = 25$, $L = 17$), each truncated term has at most $\min(\mathsf{k} + 2, 24) = 24$ significant digits, as rounding never adds digits, and the sum at most 32.

5 Rewrites over Arithmetic and Rounding

We compile an FPy specification into a model that computes with the native operations of a *target*, such as a CPU or GPU. Native operations round only to a few formats, while a specification may round to any format or compute exactly. We therefore rewrite a specification's arithmetic and its roundings until the target supports every rounded operation in it (Section 6). Our program analyses (Section 4) decide when these rewrites are semantics-preserving. We separate arithmetic from rounding with rewrites from prior work (Section 5.1) and present *novel rewrites that lower a rounding* to operations the target supports (Section 5.2). We prove each rewrite correct in Lean.

```
def mac(a, b, c) =
  with IEEE754(8, 16, RNE) in p = a * b;
  with MX_E4M3 in ret p + c
```

(a) Specification

```
def mac(a, b, c) =
  with IEEE754(8, 32, RNE) in p = a * b;
  with IEEE754(11, 64, RNE) in t = p + c;
  with MX_E4M3 in ret round(t)
```

(b) After Section 5.1

```
def mac(a, b, c) =
  with IEEE754(8, 32, RNE) in p = a * b;
  with IEEE754(11, 64, RNE) in t = p + c;
  if isnan(t) or abs(t) > 464 then
    ret nan()
  else
    n = max(logb(t) - 3, -9);
    u = 2^-n * t;
    with Fixed(0, RNE) in m = round(u);
    ret 2^n * m
```

(c) After Section 5.2

Fig. 10. A multiply-add on MX E4M3 values, after each phase of rewrites.

In this section's running example (Figure 10), `mac` multiplies two MX E4M3 values under BF16, then adds a third and rounds the result to MX E4M3, an 8-bit format of machine-learning accelerators [64]. We call `mac` under $\mathbb{R}$, so operations outside a context statement, such as those in Figure 10c, are exact. Most CPUs provide no arithmetic in BF16 or MX E4M3, so the rewrites turn `mac` into a program that uses only FP32 and FP64 arithmetic and rounding to integers.

Figure 11 lists our rewrites. Each is written $[C]\ s \rightsquigarrow s'$ if ϕ: under the current context C, it replaces s with s' when ϕ holds. The rewrite preserves semantics: whenever ϕ holds, $\langle \sigma, C, s\rangle \Downarrow_S o$ if and only if $\langle \sigma, C, s'\rangle \Downarrow_S o$ for every store $\sigma \in \gamma(\Gamma)$, up to fresh variables. Here, Γ is the environment that format inference computes at s. We state each rewrite on an assignment $x = e$; the compiler applies it wherever it can bind e to a fresh variable, such as the return value in Figure 10b.

5.1 Separating Arithmetic from Rounding

These rewrites split each operation into native arithmetic and, where needed, an explicit round (Figure 10b), which Section 5.2 then lowers. Prior work proves them correct [28, 65, 68, 71].

Name	C	s	s'	ϕ
Elim	C	`x = op(...)`	`with ℝ in x = op(`$t_1, \ldots, t_k$`)`	$op^\sharp(F_1, \ldots, F_k) \subseteq F_C$
Insert	$\mathbb{R}$	`x = op(...)`	`with `C'` in x = op(`$t_1, \ldots, t_k$`)`	$op^\sharp(F_1, \ldots, F_k) \subseteq F_{C'}$
Split	C	`x = op(...)`	`with `C'` in r = op(`$t_1, \ldots, t_k$`);` `x = round(r)`	$\mathrm{rnd}_C \circ \mathrm{rnd}_{C'} = \mathrm{rnd}_C$ on $op(F_1, \ldots, F_k)$

Special	C	`x = round(e)`	`if isnan(t) then x =` $\mathrm{rnd}_C(\mathrm{NaN})$
			`elif t == -inf then x =` $\mathrm{rnd}_C(-\infty)$
			`elif t == +inf then x =` $\mathrm{rnd}_C(+\infty)$
			`elif t == 0 then x = 0`
			`else x = round(t)`
				—

Overflow	C	`x = round(e)`	`with `C'` in r = round(t);`
			`if r <` $-b$ `then x =` ovf_C^-
			`elif r >` b `then x =` ovf_C^+
			`else x = r`
				$F_C = \mathcal{A}(p, exp, b, S)$;
				$\mathrm{fmt}(e) \subseteq \mathbb{R}$

FloatToFixed	C	`x = round(e)`	`with ℝ in`
			`  n = max(logb(t) -` p `+ 1,` exp`);`
			`with Fixed(n,` rm`) in x = round(t)`
				$F_C = \mathcal{A}(p, exp, \infty, S)$,
				$1 < p < \infty$, C uses mode rm; $\mathrm{fmt}(e) \subseteq \mathbb{R} \setminus \{0\}$

Rescale	C	`x = round(e)`	`with ℝ in u =` 2^{-n} `* t;`
			`with `C'` in r = round(u);`
			`with ℝ in x =` 2^n `* r`
				$F_C = \mathcal{A}(\infty, n, b, S)$;
				$\mathrm{rnd}_{C'}(2^{-n}v) = 2^{-n}\mathrm{rnd}_C(v)$ on $\mathbb{R}^*$

Fig. 11. Rewrites $[C]\ s \rightsquigarrow s'$ if ϕ: under C, replacing s with s' preserves semantics when ϕ holds. Each s' first binds $e_1, \ldots, e_k$ (or e) under C to fresh $t_1, \ldots, t_k$ (or t). Here, u, n, and r are fresh, C' is a rewrite-specific context, `op(...)` is $op(e_1, \ldots, e_k)$, $\mathrm{fmt}(e_i) \subseteq F_i$, and F_C is the format C induces (Section 4.1).

Eliminating and inserting roundings. A rounding has no effect when the exact result is representable in the context's format. Format inference detects such roundings, and Elim removes them, which is correct because $\mathrm{rnd}_C(v) = v$ for every $v \in F_C$. In the running example, the exact product of two MX E4M3 values has at most 8 significant digits and lies in BF16, so rounding it to BF16 has no effect. Conversely, by the same identity, Insert moves an operation from $\mathbb{R}$ to any context whose format contains the operation's result, such as one the target supports. The product `p` also lies in FP32, so FP32 multiplication computes it exactly. Likewise, in `fdpa`, Insert computes each product in FP32 (line 13 of the C++ code in Figure 2) and the sum in `fused_sum` in FP64 (line 7).

Splitting roundings. Elim and Insert need the operation's result to fit a format. When it does not, Split rounds twice, first under an intermediate context C' and then under C, which can still give the same result as rounding once under C. Its condition requires this for every possible exact result: $\mathrm{rnd}_C \circ \mathrm{rnd}_{C'} = \mathrm{rnd}_C$ on $op(F_1, \ldots, F_k)$, the set of exact results of op on inputs in $F_1, \ldots, F_k$.[12]

When $C' = \mathbb{R}$, the condition holds trivially, since s' is how E-Arith defines a rounded operation. In the running example, Split separates the addition in `mac` into an exact addition `t = p + c` and `round(t)`. The exact sum has at most 36 significant digits, so Insert then computes `t` in FP64.

Otherwise, the condition can hold in two ways. It can hold on all of $\mathbb{R}^*$, for any operation and inputs; prior work [71] gives an efficient check based on the formats, rounding modes, and overflow behavior of C and C'. For example, rounding to FP32 under RNE equals rounding first to FP64 under *round to odd* (RTO) [8]. Or it can hold only on the results of a particular operation on inputs in $F_1, \ldots, F_k$, which is a weaker requirement. When the inputs lie in F_C and both contexts round to nearest, an intermediate context with at least $2p + 2$ digits suffices for addition, subtraction,

[12] Format inference soundly approximates this set by $op^\sharp(F_1, \ldots, F_k)$.

division, and square root, where p is the precision of C [28, 65, 68].[13] For example, to compute FP16 addition under RNE, Split can add in FP32 and then round to FP16, since FP16 has 11 digits and FP32 has $24 \geq 2 \cdot 11 + 2$, even though the FP32 addition does not use RTO.

5.2 Lowering Roundings

In Figure 10b, `mac` computes `t` in FP64, but still rounds it to MX E4M3, which most CPUs cannot do. We present four novel rewrites that lower such a rounding: they replace $\text{round}(e)$ under a context C with an equivalent program that uses only operations the target supports.

Unfolding special values. A context rounds NaN and each infinity to a constant and leaves zero unchanged, so Special makes this behavior explicit with branches. It tests $-\infty$ and $+\infty$ separately because a context may round them differently. It also separates zero, which the later rewrites exclude since $\text{logb}(0) = -\infty$. Branches that format inference shows t cannot reach are omitted, and branches with the same result can be merged. In the running example, $\text{rnd}_C(\pm\infty)$ is NaN: MX E4M3 has no infinities, and `mac` uses the OCP MX specification's non-saturating overflow mode. Line 4 of Figure 10c thus handles the infinities together with NaN and overflow.

Unfolding overflow. After Special, only finite, nonzero values reach the rounding. If a context C induces a format $F_C = \mathcal{A}(p, exp, b, S)$ with $b < \infty$, rounding under C proceeds in two steps [71]: round to the unbounded format $F' = \mathcal{A}(p, exp, \infty, S)$, then return an overflow value for any result beyond b. Overflow makes these steps explicit. Here C' is C without the bound b, so it induces F', and ovf_C^- and ovf_C^+ are the values C produces on negative and positive overflow, such as $\pm\infty$, $\pm b$, or NaN. Since rounding is monotone, the test can compare t to a constant threshold before rounding. For MX E4M3 under RNE, t overflows exactly when $|t| > 464$, halfway between $b = 448$ and the next value of F'; the tie rounds to the even 448.

Floating point to fixed point. After Overflow, the remaining rounding is to F'. If F' is floating point ($p < \infty$), rounding to it keeps p significant digits. Since the leading digit of t is at position $\text{logb}(t)$ (Section 4.3), the last digit kept is at position $n = \max(\text{logb}(t) - p + 1,\ exp)$, where 2^{exp} is the format's minimum quantum. Once that position is known, floating-point rounding is *data-dependent* fixed-point rounding with quantum 2^n, which FloatToFixed makes explicit. Here $\texttt{Fixed}(n, rm)$ rounds with C's mode rm to the fixed-point format $\mathcal{A}(\infty, n, \infty, \emptyset)$. The condition $p > 1$ matters for modes that use parity, such as RNE and RTO, since one-digit formats and fixed point define parity differently. The new context depends on t, so the rewritten program computes it at run time, which FPy allows because contexts are values. In Figure 10c, line 7 computes n.

Rescaling fixed point. After FloatToFixed, or under a fixed-point context, the remaining rounding is at a fixed position n. Scaling t by 2^{-n} moves that position to 0, so Rescale turns the rounding into rounding to an integer. Its condition says that C' is C scaled by 2^{-n}, so it rounds to integers, which CPUs support directly (e.g., C++'s `std::nearbyint`). Together, FloatToFixed and Rescale reduce floating-point rounding to an exponent, a scaling, and an integer rounding (lines 7–10 of Figure 10c). In `fdpa`, Rescale turns the truncation in `fused_sum` into a scaling and a conversion to `int32_t`, which uses RTZ (lines 2–7 of the C++ code in Figure 2); digit bounds ensure each scaled term fits (Section 4.3).

6 Compiling FPy

Our compiler translates FPy to conventional languages by combining the analyses of Section 4 with the rewrites of Section 5. It first specializes each function per call site [13], so that argument formats,

[13] More precisely, addition and subtraction need $2p + 1$ digits, division $2p$, and square root $2p + 2$ [28, 65].

and contexts that do not depend on data, are known statically. It then applies these rewrites and cleans up the result by folding constants, propagating copies, and eliminating dead code. Finally, it assigns each numerical value a datatype and emits code, translating each rounded operation to a target operator. Compilation is *partial*: it fails if a rounded operation has no operator.

The compiler has two backends: one emits C++ for CPUs, and one emits Triton [76] kernels for GPUs. Each backend supplies just the operators and datatypes of its target and an emitter that expands each operator into target code. Both targets support four rounding modes for FP32 and FP64 arithmetic; Triton adds FP16 but rounds it only to nearest. The Triton backend also vectorizes loops without loop-carried dependences, evaluating each block of iterations as one tile.

We describe three parts of the compiler in more detail.

Target descriptions. A *target description* lists the operators a target provides. In C++, each operator has a type signature, such as `+ : float → float → float`, leaving its rounding implicit in the output type and the floating-point environment. We make rounding explicit: each operator names an operation of FPy, a rounding context C, the formats of its inputs, and the target code that implements it. The code computes the operation's exact result rounded under C (E-Arith), so its output lies in the format that C induces. For example, our C++ target lists `+` under `IEEE754(8, 32, rm)` on `float` inputs, implemented by `x + y`, for each rounding mode *rm* that C++ supports, and `round` under `Fixed(0, RTZ)` on a `double` input, implemented by `trunc(x)`. A rounded operation is *supported* if the target lists an operator for the same operation and context whose input formats contain the operands' formats.

Phase ordering. The compiler first applies Elim to remove unnecessary roundings. It then splits each unsupported arithmetic operation with Split and Insert, trying the target's contexts as the intermediate C', narrowest first. Each split replaces such an operation with a supported one, adding at most one rounding, so this phase terminates; if no intermediate works, compilation fails. Finally, the compiler lowers each remaining rounding by applying Special, Overflow, FloatToFixed, and Rescale once, in that order, skipping any whose condition fails.

Storage assignment. A target description also lists the datatypes that store numerical values: C++, for example, provides 8- to 64-bit signed and unsigned integers, `float`, and `double`.[14] Format inference may bound a numerical value by a format F that the target does not support, so *storage assignment* widens it to a datatype that contains F. It partitions definitions into *webs* [9], definitions of a variable linked by shared uses (e.g., in the two branches of a conditional) that must share a location, bounds each web by the join of its definitions' formats, and assigns it the first datatype that contains this bound from a fixed list, integers from smallest to largest, then floating point; if none does, compilation fails. In `fdpa`, this gives the datatypes of Figure 2. To match an operator's input datatype, the compiler inserts conversions that format inference proves lossless.

7 Evaluation

In an evaluation and case study, we answer seven research questions:

RQ1 Does the FPy compiler generate faster models than hand-written ones? (Section 7.1)
RQ2 Does the FPy compiler compile designs quickly? (Section 7.2)
RQ3 How tight are the bounds that FPy's analyses compute? (Section 7.3)
RQ4 How do the analyses and rewrites affect model performance? (Section 7.4)
RQ5 How does the choice of solver affect digit-bound analysis? (Section 7.5)
RQ6 Can FPy express the specifications of prior work? (Section 7.6)

[14] Most C++ compilers also provide `long double`, whose format varies by platform; we do not use it.

RQ7 Does the FPy compiler make design-space exploration practical? (Section 8)

We run all experiments on a machine with an AMD Ryzen Threadripper 1950X CPU, 64 GB of RAM, and an NVIDIA TITAN V GPU with 12 GB of memory, running Ubuntu 26.04.1. We use GCC 15.2, Python 3.8.10, PyTorch 2.14.0, Triton 3.8.0, and CUDA 12.6. Our evaluation uses 62 simulated designs: 49 of NVIDIA tensor cores (Volta through Blackwell), 11 of AMD matrix cores (CDNA1 through CDNA3) [44, 45], and two FMA chains. We base each FPy specification on the algorithm that Xie et al. [80] reverse-engineered for the design in MMA-Sim. Each design computes $a \cdot b + c$ for vectors a and b of length 4 to 64 and an accumulator c (Table 9).

7.1 Comparison to MMA-Sim

To answer **RQ1**, we compare the models our compiler generates with hand-written ones: MMA-Sim's bit-accurate models [80], written in PyTorch [61]. We time our C++ code on the CPU and our Triton code on the GPU, on the 60 designs FPy compiles; the other two, CDNA1's BF16 and FP16 instructions, add their products exactly and need up to 525 bits.

MMA-Sim is a behavioral model, written for clarity rather than speed. For a fair comparison, we optimize it with a few small edits that preserve its numerical behavior. MMA-Sim branches on computed values and updates tensors through boolean masks, which make the CPU wait for the GPU and break `torch.compile`'s graphs; we replace both with `torch.where`. On the CPU, we then compile it with `torch.compile` [2], which fuses its operations. On the GPU, MMA-Sim launches many small kernels per matrix product; since an LLM repeats the same shapes for every token it generates, we record each shape's kernels once as a CUDA graph and replay them with one launch, as LLM serving systems do. These changes make MMA-Sim **[X]** faster on the CPU and 1.47× faster on the GPU (median). Before timing, we check that FPy's models and MMA-Sim produce the same results.

7.1.1 Dot Products on the CPU. We time each model on batches of 1024, 4096, or 16384 independent dot products of length 1024, 4096, or 16384, nine configurations in all. A model computes each dot product by calling the design on successive slices of K elements, the design's width, passing each result to the next call as the accumulator. Inputs are drawn from $\mathcal{N}(0, 0.5^2)$ and rounded to each design's input format; block-scaled operands are quantized with TorchAO [59]. FPy's models and MMA-Sim run on **[N]** PyTorch threads but parallelize differently: ours split the batch across threads, while MMA-Sim's split each fused kernel. We time each configuration three times with PyTorch's `blocked_autorange`, each for at least 0.25 seconds, and report the median time per call, excluding input generation, quantization, correctness checks, and `torch.compile`'s compilation.

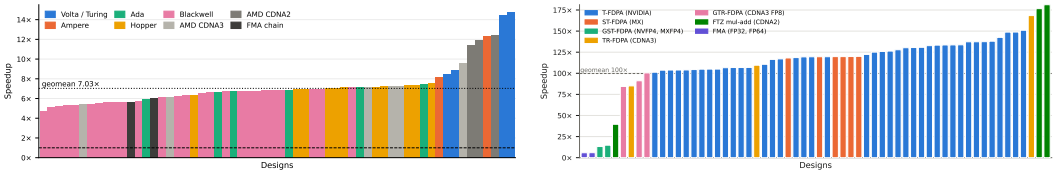


Fig. 12. Speedup of FPy over optimized MMA-Sim, per design, on the CPU (left; 16384 dot products of length 16384) and the GPU (right; the first 32 GSM8K problems, with MMA-Sim's times mostly predicted).

Results. At the largest configuration (Figure 12, left), FPy's models are faster on every design, with a speedup from 4.71× to 14.77× (geometric mean: 7.03×). The speedup depends mostly on the design's width K. Each MMA-Sim call has a fixed cost, several kernels over the whole batch,

so designs that take more calls per dot product gain the most: Volta, Turing, and Ampere's TF32 ($K = 4$ or 8) and CDNA2 (2 or 4 products per step). Blackwell's designs ($K = 32$ or 64) gain the least and the FMA chains gain little, since MMA-Sim already computes them with a native C++ FMA.

7.1.2 LLM Inference on the GPU. We ask whether FPy's models are fast enough for an LLM, Qwen3.5-0.8B [63], to solve all of GSM8K [12], 1319 grade-school math word problems. We run the model with its chat template and thinking off, using the `gsm8k_cot` task of lm-eval [75]. The task uses 8-shot chain-of-thought prompting [78] and greedy decoding of up to 256 tokens. The model's 187 linear layers multiply matrices with each design in turn, and the rest runs in FP32.

For each design, we compare the Triton kernel our compiler generates with optimized MMA-Sim on the first 32 problems, generated together in one batch. We time FPy's run end to end, including the FP32 work outside the linear layers but excluding Triton's just-in-time compilation. Running MMA-Sim end to end would take about 120 hours for all designs, so we predict its time: we time one call of each matrix-product shape the model uses, on the inputs from FPy's run, and scale each by how often the shape occurs. The prediction is within 2.6% of in-model measurements of one design per kind of operation, and within 0.1% of a full run with Hopper's BF16 design. With this design, we also measure accuracy on all 1319 problems under FPy and FP32.

Results. On the 32 timed problems, FPy is 5.8× to 181× faster than optimized MMA-Sim across the 60 designs (geometric mean: 100×; Figure 12, right). The speedup is lowest for the FMA chains and the FP4 designs (5.8–15×), whose FPy kernels are slow and whose MMA-Sim operations are cheap, and highest for CDNA2's BF16 instructions (177–181×), whose FPy kernels are the fastest. For Hopper's BF16, FPy runs all 1319 problems under lm-eval in 1.9 hours, while optimized MMA-Sim takes 2.4 hours for the first 32 alone. FPy's kernels benefit from the NVIDIA TITAN V's fast FP64 arithmetic, which most consumer GPUs lack.

On the GPU, the check of Section 7.1 finds one exception: on the FP32 FMA chain, MMA-Sim's GPU FMA flushes subnormals, while its CPU path agrees with FPy. With Hopper's BF16 arithmetic, the model answers 45.8% of the 1319 problems correctly, against 45.9% in FP32, a one-problem difference that is not statistically significant. But the design's rounding changes the exact text the model generates on 240 problems (18%), though mostly not its final answer, an effect a model must be bit-accurate to reproduce.

Bit-accurate evaluation of a design on a standard LLM benchmark thus becomes practical, because our compiler turns each design's specification into native GPU arithmetic without changing its results, e.g., turning Hopper's data-dependent truncation into integer arithmetic (Section 5.2).

7.2 Compilation Time

We time our compiler on each of the 60 designs with each backend, excluding the additional time due to GCC and Triton compilation. The C++ backend compiles each design alone (Section 7.1.1), while the Triton backend compiles it inside a matrix-multiply loop (Section 7.1.2).

Figure 13 shows the results. The median design compiles in 6.8 seconds to C++ and 7.1 seconds to Triton. While most designs compile in under 10 seconds, the three CDNA3 FP8 designs take up to 22 seconds to compile. This slowdown is due to digit-bound analysis, which generates 12,013 constraints and 126 solver queries for CDNA3 FP8, compared to the median of 5,035 and 38, respectively. In answer to **RQ2**, the compilation time for each evaluated design is on the order of seconds. This is likely reasonable for most use cases as the compilation time for the generated C++ programs by GCC is also on the order of seconds.

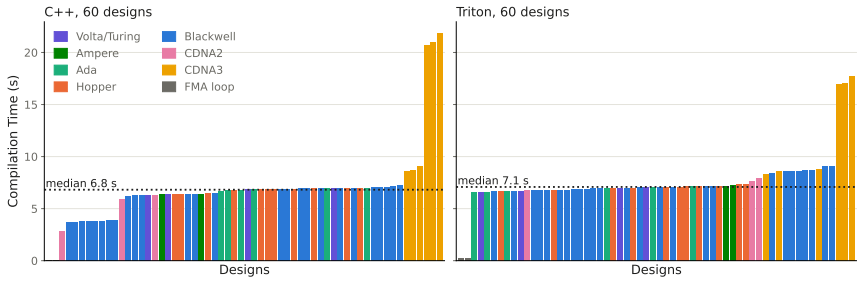


Fig. 13. Time to compile each design, with each backend.

7.3 Tightness of Inferred Bounds

The compiler chooses rewrites and datatypes from the bounds our analyses infer (Section 4), so we measure how tight these bounds are compared to observed values. We analyze the 62 MMA-Sim specifications of Section 7.1 after the compiler specializes each call but before applying the rewrites of Section 5. A *site* is an assignment in the specialized specifications; there are 697. We analyze each specification first with format inference alone (FI) and then with digit-bound analysis as well. We run each specification on 5000 inputs with the FPy interpreter, recording the values assigned at each site. Of the inputs, 4000 are random, drawing each value from its format, sometimes a special case such as ± 0 or $\pm\infty$; the other 1000 are *adversarial*, setting every operand to an extreme of its format, such as all tiny or all huge.

At each site, we compare the inferred format $\mathcal{A}(p, exp, b)$ with the least format $\mathcal{A}(p', exp', b')$ containing all recorded values, ignoring special values. The *tightness* of each parameter is a ratio: p'/p for precision,[15] $2^{exp}/2^{exp'}$ for the minimum quantum, and b'/b for the maximum bound; it is 100% when the parameter is attained and 0% when the quantum or bound is unbounded. Since we sample inputs, measured tightness is a lower bound on the true tightness.

Results. In Table 1, format inference alone bounds 143 of the 697 sites (21%) only by $\mathbb{R}^*$, including the truncated terms of every NVIDIA design, whose digits depend on `emax` (Figure 5). Digit-bound analysis bounds all of them, but six, one per CDNA3 design, lack a maximum bound (tightness 0%): each rounds the final sum at a position computed from the sum itself, and the analysis bounds that position only from below.

Table 1. Precision of the analyses: the number of sites where format inference (FI) alone infers only $\mathbb{R}^*$, and the mean tightness of each inferred parameter over the 692 sites with a nonzero observed value.

	Volta	Turing	Ampere	Ada	Hopper	Blackwell	CDNA1	CDNA2	CDNA3	FMA	Total
Designs	2	2	2	6	11	26	2	3	6	2	62
Sites	22	22	22	66	121	266	16	53	105	4	697
$\mathbb{R}^*$ sites (FI)	4	4	4	12	22	52	0	0	45	0	143
p'/p	99%	99%	99%	98%	97%	97%	97%	100%	97%	100%	97%
$2^{exp}/2^{exp'}$	91%	91%	82%	83%	87%	95%	100%	85%	85%	100%	90%
b'/b	94%	92%	93%	91%	89%	87%	90%	91%	66%	100%	86%

On average, observed values carry 97% of the digits that the inferred precision allows.[16] Most remaining looseness comes from our inputs, not the analyses. Reaching the minimum quantum of a

[15] Abstract formats are redundant [71]: different precisions can give the same values; we take the least.

[16] Over the 692 sites with a nonzero observed value; of the other 5, 3 never run and 2 only ever assign zero.

truncated sum needs every product to be zero with an accumulator just above the smallest normal value, and reaching the maximum bound needs every term to have a maximal significand and the same sign; our inputs produce neither. Besides the CDNA3 maximum bounds, the analyses lose precision in one case: specifications give a zero product a very small exponent, so the truncation position can be very low, though only when every product is zero, and the truncated terms then have no digits. The linear constraints of digit-bound analysis cannot express this case split, so on sums with an FP32 accumulator, the inferred minimum quantum is about 2^7 times too small.

In answer to **RQ3**, the combination of analyses is precise in practice. As a check on our implementation, no observed value lies outside its inferred format in any run.

7.4 RQ4: Ablation

To measure what each part of the compiler contributes, we compile each of the 62 designs to C++ in eight configurations: with format inference alone (FI) or with digit-bound analysis (FI+DB), and with no rewrites, the rewrites of Section 5.1, those of Section 5.2, or both. The rewrites of Section 5.1 reduce to Elim and Insert because none of the designs use Split.

Table 2. Number of designs that compile to C++ (out of 62), for each combination of analyses (rows) and rewrites (columns).

Analyses	None	Section 5.1	Section 5.2	Section 5.1 + Section 5.2
FI	2	5	2	5
FI+DB	2	5	2	60

Table 2 shows the results. The two CDNA1 designs compile in no configuration, since FPy determines their exact sums need up to 524 significand bits. The two FMA chains compile in every configuration, since they use only FP32 and FP64 arithmetic. The three CDNA2 designs additionally need Insert, which computes their exact operations in FP32. The remaining NVIDIA and CDNA3 designs round their truncated terms to a runtime-computed position, therefore needing all rewrites and analyses. Format inference alone cannot relate a term to its position, so it bounds the term by $\mathbb{R}^*$ (Section 7.3). This bound needs up to 538 significand bits once the rounding is lowered, wider than any C++ datatype. With digit-bound analysis, Section 5.2 lowers the run-time roundings and Insert computes the exact arithmetic in C++. Thus the rewrites and analyses determine which designs compile.

7.5 RQ5: Sensitivity to the Solver Backend

Digit bound analysis (section 4.3) uses a constraint solver to bound the number of digits each expression can take. To measure how sensitive digit bound analysis is to the choice of constraint solver, we compare three solver backends: the Z3 [21] and cvc5 [3] SMT solvers using binary search and Z3 using Optimize, its built-in optimizer.

Methodology. We implement a solver interface for digit bound analysis which allows the backend to be swapped out. This interface has two operations: *assume*, which adds a constraint, and *maximize*, which returns an upper bound on each of a list of terms under the constraints so far. The interface splits the constraints into independent components, and answers each query using only the component its terms belong to. The binary-search backends find each term's maximum by bisecting over satisfiability checks and Optimize answers all of a query's terms in one optimization call. We do not evaluate using an ILP solver: ILP solvers solve in floating-point arithmetic, so a maximum they return can be slightly below the true one, making digit bound analysis unsound.

We instrument each backend to record the time spent in the solver, the number of queries, and the size of the component each query is asked of. We report the median of 3 runs per design, running 4 compilations in parallel, with a 60 s per-call limit. We use Z3 version 5.3.3 and cvc5 version 1.4.2.

Table 3. Compile time and solver time for each solver backend, summed over the 60 designs that compile. Solver share is the percentage of compile time spent in the solver. Time/query is solver time divided by the number of queries (15,336).

Backend	Solver method	Compile	Solver	Solver share	Time/query
Z3	Binary search	361 s	133 s	37%	8.6 ms
Z3	Optimize	461 s	235 s	51%	15.3 ms
cvc5	Binary search	438 s	215 s	49%	14.0 ms

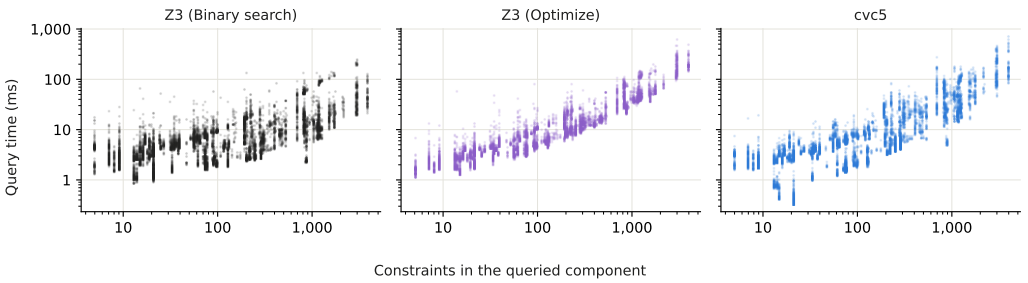


Fig. 14. Query time vs. number of constraints in the queried component, one panel per solver backend, on log–log axes. Each point is one query from the median run of each of the 60 designs. The 814 queries over no constraints are not shown.

Results. Table 3 shows compile time and solver time for each backend, summed across the 60 designs that compile. Z3 with binary search is the fastest, spending 133 s in the solver, which accounts for 37% of total compile time. The other backends spend 1.6–1.8× as long in the solver: cvc5 215 s (49%), and Optimize 235 s (51%). Every backend computes the exact maximum and no call times out, so the generated C++ is identical between the solvers.

Figure 14 plots each query's time against the number of constraints in its component. For every backend, query time grows with component size. Below 100 constraints, the backends take a similar amount of time, each taking a few milliseconds. On components with over 1,000 constraints, the median query for Optimize and cvc5 takes about 3× as long as Z3 with binary search.

Thus, changing the solver backend changes compile time by at most 1.3× and never changes the generated code, so digit bound analysis does not depend on a particular SMT solver.

7.6 Expressiveness

To display the expressiveness of FPy, we port FPBench [17], the standard suite for floating-point accuracy benchmarks. A translator parses and converts all 136 FPCore programs into a FPy function. This process preserves the precondition and all other annotations as specified in the file. Every FPCore program in this suite is straightforwardly expressed as an FPy program.

We validate each port by differential testing against Titanic [82], an FPCore interpreter. Each port's result and precondition agree with the original's on 1000 inputs.

8 Case Study: Design-Space Exploration of Tensor Cores

To answer **RQ7**, we follow Figure 1 end to end: a generator proposes candidate designs (*Explore*), our compiler turns each into a model (*Compile*), and a harness evaluates each model on an application (*Evaluate*). As a proxy for a real hardware design loop, we explore variants of the numerical specifications of *existing* NVIDIA and AMD tensor cores and evaluate them in LLM inference and general matrix multiplications (GEMMs).

Explore. We write a generator that produces FPy specifications of candidate designs for a dot-product-accumulate instruction based on the BF16, FP8, and MXFP8 instructions of NVIDIA's Ampere, Ada, Hopper, and Blackwell GPUs and AMD's CDNA2 and CDNA3 GPUs. We base our specifications on open-source reverse-engineering of NVIDIA and AMD tensor cores [80].[17]

Our specifications vary at each stage of the dot product in Figure 2: the *input* formats, whose elements may share a scale factor per 32, as in OCP MX [64]; the *products*, exact or rounded with subnormals flushed to zero; the *summation*, of some number of products, fused and truncated to k bits or a tree of rounded additions; when `c` *is added*, in the fused sum or after it, rounding `c` and the sum toward negative infinity; and the *output* context `out`, which sets the accumulator's format and rounding. We generate 113 designs, each either one of 8 existing instructions or a variant of one that differs in the truncation width k, in when `c` is added, or in both.

Compile. Each specification models one instruction, a fixed-length dot product; a matrix product computes each entry by calling it on successive slices of a row and a column, passing each result to the next call as `c`. For LLM inference, we write this loop in FPy and compile it with the specification to a Triton kernel (Section 6). Our GPU does not support BF16 and FP8 arithmetic, so the kernels compute with FP16, FP32, FP64, and 8- to 64-bit integers. For GEMMs, we write the loop in C++ around the specification compiled to C++, adding no rounding of its own. Our compiler compiles 105 of the 113 designs in 81 seconds to C++ across 32 processes. It rejects CDNA3 FP8 variants that add `c` with the products: our analyses cannot infer a tight enough bound to store the sum in a machine datatype.

8.1 Evaluate: LLM Inference

Tensor cores mainly serve machine learning, so we measure how each design changes the predictions of an LLM, Qwen3.5-0.8B [63]. We run it in Hugging Face Transformers [79], computing every linear layer's matrix product with the design's kernel and the rest of the model in single precision. Each layer quantizes its weights and input to the design's formats: BF16, E4M3 scaled per row, or, for Blackwell's FP8 instruction, MXFP8. The reference multiplies the same quantized inputs in double precision and rounds the product once to single precision, so differences come from the design's arithmetic alone. We measure the Kullback–Leibler (KL) divergence [46] of each design's next-token distribution from the reference's, averaged over 10 random 2048-token segments of WikiText-2 [52]. Unlike accuracy on a benchmark, KL divergence detects small changes in a model's predictions [26]. For comparison, we also measure the KL divergence when every output of the reference's matrix products is off by one unit in the last place (ULP), up or down at random.

We run each design two ways. The *observed* run mirrors deployment: each layer takes an input x and a quantized weight matrix W, quantizes x to $\hat{x}$, and computes $\hat{x}W^\top$ with the design's kernel. However, quantizing x introduces more error than the design does, so we add an *amplified* run, in which each layer instead takes the reference's input x_r, quantizes it to $\hat{x}_r$, computes $\hat{x}_r W^\top$, and adds a correction term $(x - x_r)W^\top$ for the error of earlier layers.

[17] Each existing instruction's specification matches MMA-Sim, and its kernel the FPy interpreter, bit for bit on 1000 inputs.

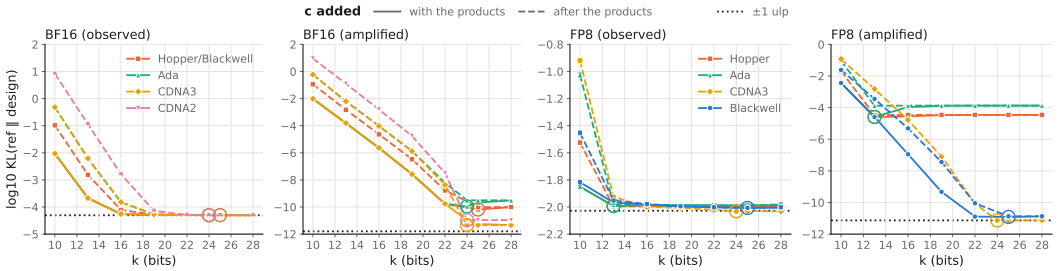


Fig. 15. KL divergence of each design's next-token predictions from the reference's, against truncation width k. Line style shows when c is added; circles mark existing instructions.

Results. Figure 15 plots each design's KL divergence against its truncation width k; lower KL divergence means predictions closer to the reference's. Within each subplot, the designs that are Pareto-optimal in k and KL divergence lie toward the lower left.

Evaluating the 105 models takes **[T]** on our GPU, fast enough to answer design questions such as the following. *How wide must k be?* In the observed run, every design reaches the ±1 ULP line by about 19 bits for BF16 inputs and 13 for FP8, while existing BF16 instructions truncate to 24 or 25 bits. In the amplified run, KL divergence stops falling near $k = 24$. *Would a wider k help Ada's and Hopper's FP8 instructions?* Not on its own: in the amplified run, they level off at $k = 13$. Since each instruction keeps only 13 of the 23 fraction bits of its FP32 result [44, 80] and passes it to the next as c, the accumulator's precision, not k, limits their accuracy. *When should* c *be added?* For small k, adding it with the products is more accurate in every subplot than adding it after them.

8.2 Evaluate: GEMM Accuracy

Tensor cores also accelerate numerical libraries, so we measure each design's worst-case accuracy on GEMMs: with the GEMM loop, we multiply the 1024×1024 matrices of Tests 1c and 2c of Demmel et al. [24], with entries rounded to the design's input format. These tests, designed for BLAS routines, are also difficult for tensor core instructions: Test 1c hides a tiny product among products of zero and huge values, and Test 2c sums products whose magnitudes range from about 2^{-2s} to 2^{2s}, for scales s across the input format's range. For each design, we report the worst componentwise backward error $\eta = \max_{i,j} |\hat{C}_{ij} - C_{ij}|/(|A|\,|B|)_{ij}$ over both tests and the entries that Demmel et al. [24] score, where $C = AB$ is the exact product (computed in double precision) and $\hat{C}$ is the computed product. For comparison, we also measure η for a linear chain of FP32 FMAs.

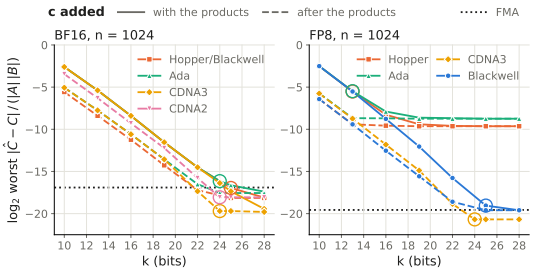


Fig. 16. Worst componentwise backward error η of each design, against truncation width k. Line style shows when c is added; circles mark existing instructions; *FMA* is the worst case of chained FP32 FMAs.

Results. Figure 16 plots each design's worst componentwise backward error η against its truncation width k; lower η means a more accurate product. Measuring η for the 105 designs takes **[T]**, fast enough to ask the questions of Section 8.1 again. For BF16 inputs, every existing instruction is within a factor of 2 of the FMA chain's η, and the CDNA2 and CDNA3 instructions are below it **[verify against the data]**. The FP8 instructions for Ada and Hopper ($k = 13$) have $\eta \approx 2^{-5.5}$, about 2^{14} times the FMA chain's. Widening k does not fix them: as in Section 8.1, their 13-bit accumulator limits them, consistent with reports for Hopper [23]. The Blackwell and CDNA3 instructions improve with k and get close to the FMA chain's η at their own widths. Unlike in Section 8.1, adding `c` after the products is more accurate for narrow k on these tests: at $k = 10$, it lowers η by $2^{2.4}$ to $2^{3.9}$ for every generation, and the two orders meet by $k = 28$.

9 Related Work

We compare FPy with prior work on compiling specifications, modeling hardware, specifying rounding, static analysis, and program rewriting.

Compiling numerical specifications. Keding et al. [42] compile fixed-point designs with static formats from the FRIDGE environment [43] into bit-accurate models that compute with native integers. FPy supports fixed-point, floating-point, and MX formats and data-dependent rounding, and compiles to native floating-point as well as integer arithmetic. iFP [25] compiles FPCore [16] programs with static precision annotations into bit-accurate C++ over a parameterized library; our models compute with native arithmetic.

Bit-accurate models of numerical hardware. Hardware vendors build bit-accurate models in-house [69], often as C++ "golden models"; public models instead reverse-engineer existing hardware. MMA-Sim [80] models NVIDIA and AMD matrix units in Python, and Khattak et al. [44, 45] model them in MATLAB, using CPFloat [27] for low-precision formats. Valpey et al. [77] formalize three generations of NVIDIA tensor cores in SMT to generate inputs that distinguish them. Arbitor [40] emulates DNN accelerators with user-written implementations of each number format. Each of these models is written by hand. While FPy specifications are also written by hand, our compiler generates models from them that compute with native arithmetic.

Specifying rounding. FPy is inspired by FPCore [16, 30], whose rounding contexts also control how each operation rounds; however, they are static, while FPy's are first-class values to support data-dependent rounding. In C and C++ [38], an operation's rounding is determined by its operands' types, such as `float`, and by the dynamic floating-point environment, such as the rounding mode; in FPy, the current rounding context alone determines it. Other systems support rounding contexts via libraries: Python's `decimal` [62] and APyTypes [34] provide Python context managers that install a context for a block. However, `decimal`'s contexts cover only decimal floating point, and APyTypes' set only the rounding mode; FPy's contexts determine both format and rounding.

Abstract interpretation. Interval analysis [14] is the classic abstract interpretation of numerical programs, bounding each value's range; bit-width analysis [74] does the same for hardware integers. Format inference [71], which we extend into an abstract interpretation, bounds a value's quantum and precision as well as its range. Martel [48] also infers floating-point formats, choosing the least precision each variable needs to meet an accuracy requirement on the result; we instead bound the values a program produces under its rounding. Gappa [20], a prover rather than a static analysis, reasons about range, quantum, and precision, but only to prove user-stated bounds on straight-line expressions. Miné [55] adds abstract domains to Astrée [6] that track the bits of machine integers and floats; formats instead bound which digits of a value can be nonzero.

Constraint-based analysis. Formats bound a value's digit positions only in absolute terms, so format inference loses precision when rounding depends on data (Section 4.3). Digit-bound analysis bounds these positions with linear constraints, as several prior analyses do. Adjé et al. [1] tune

precision: given an accuracy requirement on a program's result, they choose the fewest significant bits for each value by solving linear constraints, with each value's magnitude fixed by a range analysis. Digit-bound analysis does not change a program's precision; it bounds each value's precision, quantum, and magnitude, even when its digits' positions depend on program values. Zhang and Aiken [81] prove error bounds for algorithms such as compensated summation and double-double addition, abstracting each floating-point number by its sign, exponent, and leading and trailing zeros and ones, and relating these with linear constraints. Their abstraction is finer than ours for numbers in a fixed format; we instead bound values with no fixed format, including digit positions computed by the program. Rugina and Rinard [67] bound the array indices a program accesses by polynomials in its parameters, solving the constraints as a linear program; we instead bound digit positions, in terms of values such as a truncation position. Most digit-bound constraints have the form $x - y \leq c$, as in zones and octagons [53, 54], but our constraint language (Figure 8) also has sums of arbitrarily many terms, maxima, and minima, which these domains cannot represent.

Semantics-preserving rewriting. Prior work [71] decides when roundings can be dropped, inserted, or modified without changing a program's results. Results on double rounding answer this question for particular pairs of rounding modes [8] or particular operations [28, 65, 68], and libraries such as chop [35] and CPFloat [27] apply them by hand. Other program rewriters, such as superoptimizers [49, 72] and Alive [47, 51], search for or verify equivalent programs, including floating-point code; these rewrites are restricted to machine types. Traditional compilers, such as GCC, Clang, and CompCert [7], preserve floating-point semantics only for the types their languages provide. When the target lacks one of these types, compilers implement it as a special case: GCC, for example, can compute FP16 operations in FP32 and round each result [31]. Our rewrites apply to roundings in any format, including data-dependent ones, and lower them to native arithmetic when our analyses show that their conditions hold; we prove each one correct in Lean.

Approximate rewriting. Other tools rewrite programs in ways that change their results. Herbie [60, 70] and Salsa [15] rewrite floating-point expressions and programs to improve accuracy or speed, treating the real-valued result as the specification. STOKE-FP [73] adapts the STOKE superoptimizer to floating-point code, accepting rewrites within a user-specified error bound. Precision tuners [5, 11, 18, 66] lower the precision of individual variables or operations while keeping the result within a user-specified error bound; Isychev and Lohar [39] give an overview. Rosa [19] compiles real-valued programs to finite precision, and RealCake [4] verifies fast-math optimizations, both guaranteeing results within an error bound of the real-valued result. SeeDot [32] and TAFFO [10] compile real-valued and floating-point programs to fixed point, approximating their results. Our compiler instead preserves a specification's results exactly, since a model must match the hardware.

10 Conclusion

We present FPy, an executable specification language that makes every rounding explicit, with first-class rounding contexts for data-dependent rounding. We formulate format inference as an abstract interpretation over programs with loops, conditionals, and calls, and introduce a constraint-based digit-bound analysis that bounds significant digits relative to program values. Checked by these analyses, semantics-preserving rewrites over arithmetic and rounding lower each rounding to native arithmetic, so our compiler turns each specification into a fast, bit-accurate model.

AI Use Statement

We wrote the definitions and theorem statements in Lean by hand and used Claude to mechanize their proofs. We initially developed FPy by hand; later, Claude Code implemented our design plans and tests, and we reviewed this code. We used Claude Opus 4.7, 4.8, 5, and 5.5.

Data-Availability Statement

Our anonymized supplementary material contains the Lean 4 mechanization of Sections 4 and 5 and the appendices, which map each result to its Lean theorem. The mechanization uses only Lean's standard axioms. For Artifact Evaluation, we will submit the source code of FPy and its compiler and the scripts required to re-run the evaluation (Section 7).

References

[1] Assalé Adjé, Dorra Ben Khalifa, and Matthieu Martel. 2021. Fast and Efficient Bit-Level Precision Tuning. In *Static Analysis (SAS) (Lecture Notes in Computer Science, Vol. 12913)*. Springer, 1–24. doi:10.1007/978-3-030-88806-0_1

[2] Jason Ansel, Edward Yang, Horace He, Natalia Gimelshein, Animesh Jain, Michael Voznesensky, Bin Bao, Peter Bell, David Berard, Evgeni Burovski, Geeta Chauhan, Anjali Chourdia, Will Constable, Alban Desmaison, Zachary DeVito, Elias Ellison, Will Feng, Jiong Gong, Michael Gschwind, Brian Hirsh, Sherlock Huang, Kshiteej Kalambarkar, Laurent Kirsch, Michael Lazos, Mario Lezcano, Yanbo Liang, Jason Liang, Yinghai Lu, C. K. Luk, Bert Maher, Yunjie Pan, Christian Puhrsch, Matthias Reso, Mark Saroufim, Marcos Yukio Siraichi, Helen Suk, Shunting Zhang, Michael Suo, Phil Tillet, Xu Zhao, Eikan Wang, Keren Zhou, Richard Zou, Xiaodong Wang, Ajit Mathews, William Wen, Gregory Chanan, Peng Wu, and Soumith Chintala. 2024. PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph Compilation. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2* (La Jolla, CA, USA) *(ASPLOS '24)*. Association for Computing Machinery, New York, NY, USA, 929–947. doi:10.1145/3620665.3640366

[3] Haniel Barbosa, Clark Barrett, Martin Brain, Gereon Kremer, Hanna Lachnitt, Makai Mann, Abdalrhman Mohamed, Mudathir Mohamed, Aina Niemetz, Andres Nötzli, Alex Ozdemir, Mathias Preiner, Andrew Reynolds, Ying Sheng, Cesare Tinelli, and Yoni Zohar. 2022. cvc5: A Versatile and Industrial-Strength SMT Solver. In *Tools and Algorithms for the Construction and Analysis of Systems: 28th International Conference, TACAS 2022, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2022, Munich, Germany, April 2–7, 2022, Proceedings, Part I*. Springer-Verlag, Berlin, Heidelberg, 415–442. doi:10.1007/978-3-030-99524-9_24

[4] Heiko Becker, Robert Rabe, Eva Darulova, Magnus O. Myreen, Zachary Tatlock, Ramana Kumar, Yong Kiam Tan, and Anthony Fox. 2022. Verified Compilation and Optimization of Floating-Point Programs in CakeML. In *European Conference on Object-Oriented Programming (ECOOP)*. LIPIcs. https://drops.dagstuhl.de/opus/volltexte/2022/16229/pdf/LIPIcs-ECOOP-2022-1.pdf

[5] Sofiane Bessaï, Dorra Ben Khalifa, Hanane Benmaghnia, and Matthieu Martel. 2022. Fixed-Point Code Synthesis Based on Constraint Generation. In *Design and Architecture for Signal and Image Processing*, Karol Desnos and Sergio Pertuz (Eds.). Springer International Publishing, Cham, 108–120.

[6] Bruno Blanchet, Patrick Cousot, Radhia Cousot, Jérôme Feret, Laurent Mauborgne, Antoine Miné, David Monniaux, and Xavier Rival. 2003. A static analyzer for large safety-critical software. In *Proceedings of the ACM SIGPLAN 2003 Conference on Programming Language Design and Implementation 2003, San Diego, California, USA, June 9-11, 2003*, Ron Cytron and Rajiv Gupta (Eds.). ACM, 196–207. doi:10.1145/781131.781153

[7] Sylvie Boldo, Jacques-Henri Jourdan, Xavier Leroy, and Guillaume Melquiond. 2015. Verified Compilation of Floating-Point Computations. *Journal of Automated Reasoning* 54, 2 (2015), 135–163. http://xavierleroy.org/publi/floating-point-compcert.pdf

[8] Sylvie Boldo and Guillaume Melquiond. 2005. When double rounding is odd. In *17th IMACS World Congress*. Paris, France, 11. https://inria.hal.science/inria-00070603

[9] Preston Briggs, Keith D. Cooper, and Linda Torczon. 1994. Improvements to Graph Coloring Register Allocation. *ACM Transactions on Programming Languages and Systems* 16, 3 (1994), 428–455. doi:10.1145/177492.177575

[10] Stefano Cherubin, Daniele Cattaneo, Michele Chiari, Antonio Di Bello, and Giovanni Agosta. 2020. TAFFO: Tuning Assistant for Floating to Fixed Point Optimization. *IEEE Embed. Syst. Lett.* 12, 1 (2020), 5–8. doi:10.1109/LES.2019.2913774

[11] Wei-Fan Chiang, Mark Baranowski, Ian Briggs, Alexey Solovyev, Ganesh Gopalakrishnan, and Zvonimir Rakamarić. 2017. Rigorous floating-point mixed-precision tuning. *SIGPLAN Not.* 52, 1 (Jan. 2017), 300–315. doi:10.1145/3093333.3009846

[12] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. Training Verifiers to Solve Math Word Problems. arXiv:2110.14168 [cs.LG] https://arxiv.org/abs/2110.14168

[13] Keith D Cooper, Mary W Hall, and Ken Kennedy. 1993. A methodology for procedure cloning. *Comput. Lang.* 19, 2 (April 1993), 105–117. doi:10.1016/0096-0551(93)90005-L

[14] Patrick Cousot and Radhia Cousot. 1977. Abstract interpretation: a unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *Proceedings of the 4th ACM SIGACT-SIGPLAN Symposium on Principles*

of Programming Languages (Los Angeles, California) *(POPL '77)*. Association for Computing Machinery, New York, NY, USA, 238–252. doi:10.1145/512950.512973

[15] Nasrine Damouche and Matthieu Martel. 2018. Salsa: An Automatic Tool to Improve the Numerical Accuracy of Programs. In *Automated Formal Methods (AFM 2017) (Kalpa Publications in Computing, Vol. 5)*. EasyChair, 63–76. doi:10.29007/j2fd

[16] Nasrine Damouche, Matthieu Martel, Pavel Panchekha, Chen Qiu, Alexander Sanchez-Stern, and Zachary Tatlock. 2017. Toward a Standard Benchmark Format and Suite for Floating-Point Analysis. In *Numerical Software Verification*, Sergiy Bogomolov, Matthieu Martel, and Pavithra Prabhakar (Eds.). Springer International Publishing, Cham, 63–77.

[17] Nasrine Damouche, Matthieu Martel, Pavel Panchekha, Chen Qiu, Alexander Sanchez-Stern, and Zachary Tatlock. 2017. Toward a Standard Benchmark Format and Suite for Floating-Point Analysis. In *Numerical Software Verification*, Sergiy Bogomolov, Matthieu Martel, and Pavithra Prabhakar (Eds.). Springer International Publishing, Cham, 63–77. doi:10.1007/978-3-319-54292-8_6

[18] Eva Darulova, Einar Horn, and Saksham Sharma. 2018. Sound mixed-precision optimization with rewriting. In *Proceedings of the 9th ACM/IEEE International Conference on Cyber-Physical Systems* (Porto, Portugal) *(ICCPS '18)*. IEEE Press, 208–219. doi:10.1109/ICCPS.2018.00028

[19] Eva Darulova and Viktor Kuncak. 2017. Towards a Compiler for Reals. *ACM Trans. Program. Lang. Syst.* 39, 2, Article 8 (2017), 27 pages. doi:10.1145/3014426

[20] Marc Daumas and Guillaume Melquiond. 2010. Certification of bounds on expressions involving rounded operators. *ACM Trans. Math. Softw.* 37, 1, Article 2 (Jan. 2010), 20 pages. doi:10.1145/1644001.1644003

[21] Leonardo De Moura and Nikolaj Bjørner. 2008. Z3: an efficient SMT solver. In *Proceedings of the Theory and Practice of Software, 14th International Conference on Tools and Algorithms for the Construction and Analysis of Systems* (Budapest, Hungary) *(TACAS'08/ETAPS'08)*. Springer-Verlag, Berlin, Heidelberg, 337–340.

[22] Leonardo de Moura and Sebastian Ullrich. 2021. The Lean 4 Theorem Prover and Programming Language. In *Automated Deduction – CADE 28*. Springer International Publishing, 625–635. doi:10.1007/978-3-030-79876-5_37

[23] DeepSeek-AI. 2025. DeepSeek-V3 Technical Report. arXiv:2412.19437 [cs.CL] https://arxiv.org/abs/2412.19437

[24] James Demmel, Greg Henry, Igor Kozachenko, Julien Langou, Xiaoye Sherry Li, Jason Riedy, and Jackson Vanover. 2026. How to grade the accuracy of the BLAS. arXiv:2609.12307 [math.NA] https://arxiv.org/abs/2609.12307

[25] Theo Drane, Bill Zorn, and Samuel Coward. 2023. Novel Numerical Hardware Design Methodology – From Machine Readable Specification to Optimized RTL. In *Workshop on Programming Languages for Architecture (PLARCH)*.

[26] Abhinav Dutta, Sanjeev Krishnan, Nipun Kwatra, and Ramachandran Ramjee. 2024. Accuracy is Not All You Need. In *Advances in Neural Information Processing Systems*, A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang (Eds.), Vol. 37. Curran Associates, Inc., 124347–124390. doi:10.52202/079017-3950

[27] Massimiliano Fasi and Mantas Mikaitis. 2023. CPFloat: A C Library for Simulating Low-precision Arithmetic. *ACM Trans. Math. Softw.* 49, 2, Article 18 (June 2023), 32 pages. doi:10.1145/3585515

[28] Samuel A. Figueroa. 1995. When is double rounding innocuous? *SIGNUM Newsl.* 30, 3 (July 1995), 21–26. doi:10.1145/221332.221334

[29] Laurent Fousse, Guillaume Hanrot, Vincent Lefèvre, Patrick Pélissier, and Paul Zimmermann. 2007. MPFR: A multiple-precision binary floating-point library with correct rounding. *ACM Trans. Math. Softw.* 33, 2 (June 2007), 15 pages. doi:10.1145/1236463.1236468

[30] FPBench Developers. 2020. *FPCore 2.0 Specification.* Retrieved October 31, 2026 from https://fptalks.org/spec/fpcore-2.0.html

[31] Free Software Foundation. 2026. *Using the GNU Compiler Collection: Half-Precision Floating Point.* Retrieved October 31, 2026 from https://gcc.gnu.org/onlinedocs/gcc/Half-Precision.html

[32] Sridhar Gopinath, Nikhil Ghanathe, Vivek Seshadri, and Rahul Sharma. 2019. Compiling KB-sized machine learning models to tiny IoT devices. In *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI 2019, Phoenix, AZ, USA, June 22-26, 2019*, Kathryn S. McKinley and Kathleen Fisher (Eds.). ACM, 79–95. doi:10.1145/3314221.3314597

[33] John R. Hauser. 2018. *Berkeley SoftFloat: A Software Implementation of IEEE Floating-Point Arithmetic.* Conforms to the IEEE Standard for Binary Floating-Point Arithmetic. https://www.jhauser.us/arithmetic/SoftFloat.html

[34] Mikael Henriksson, Theodor Lindberg, and Oscar Gustafsson. 2024. APyTypes: Algorithmic Data Types in Python for Efficient Simulation of Finite Word-Length Effects. In *2024 IEEE 31st Symposium on Computer Arithmetic (ARITH)*. 72–75. doi:10.1109/ARITH61463.2024.00021

[35] Nicholas J. Higham and Srikara Pranesh. 2019. Simulating Low Precision Floating-Point Arithmetic. *SIAM J. Sci. Comput.* 41, 5 (2019), C585–C602. doi:10.1137/19M1251308

[36] IEEE 2019. *IEEE Standard for Floating-Point Arithmetic.* Technical Report IEEE Std 754-2019 (Revision of IEEE 754-2008). IEEE. 1–84 pages. doi:10.1109/IEEESTD.2019.8766229

[37] IEEE SA P3109 Working Group. 2026. *Interim Report on Binary Floating-point Formats for Machine Learning*. Technical Report. Institute of Electrical and Electronics Engineers (IEEE). © 2026 by The Institute of Electrical and Electronics Engineers, Inc.. https://github.com/P3109/Public/blob/main/Shared%20Reports/IEEE%20WG%20P3109%20Interim%20Report.pdf

[38] ISO/IEC JTC1/SC22/WG21. 2023. Working Draft, Standard for Programming Language C++. https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/n4950.pdf. N4950.

[39] Anastasia Isychev and Debasmita Lohar. 2025. Cost of Soundness in Mixed-Precision Tuning. *Proc. ACM Program. Lang.* 9, OOPSLA2, Article 359 (Oct. 2025), 28 pages. doi:10.1145/3763137

[40] Chenhao Jiang, Anand Jayarajan, Hao Lu, and Gennady Pekhimenko. 2023. Arbitor: A Numerically Accurate Hardware Emulation Tool for DNN Accelerators. In *2023 USENIX Annual Technical Conference (USENIX ATC 23)*. USENIX Association, Boston, MA, 519–536. https://www.usenix.org/conference/atc23/presentation/jiang-chenhao

[41] Andreas Kalaitzis et al. 2018. The bfloat16 floating-point format. *Google AI Blog* (2018). https://cloud.google.com/blog/products/ai-machine-learning/bfloat16-the-secret-to-high-performance-on-cloud-tpus

[42] Holger Keding, Martin Coors, Olaf Lüthje, and Heinrich Meyr. 2001. Fast Bit-True Simulation. In *Proceedings of the 38th Design Automation Conference, DAC 2001, Las Vegas, NV, USA, June 18-22, 2001*. ACM, 708–713. doi:10.1145/378239.379052

[43] Holger Keding, Markus Willems, Martin Coors, and Heinrich Meyr. 1998. FRIDGE: A Fixed-Point Design and Simulation Environment. In *1998 Design, Automation and Test in Europe (DATE '98), February 23-26, 1998, Le Palais des Congrès de Paris, Paris, France*, Patrick M. Dewilde, Franz J. Rammig, and Gerry Musgrave (Eds.). IEEE Computer Society, 429–435. doi:10.1109/DATE.1998.655893

[44] Faizan Khattak and Mantas Mikaitis. 2026. Accurate Models of NVIDIA Tensor Cores. *ACM Trans. Archit. Code Optim.* 23, 3, Article 105 (Aug. 2026), 25 pages. doi:10.1145/3830409

[45] Faizan A Khattak, Mantas Mikaitis, and Carlo J. Graziani. 2026. Accurate Models of AMD Matrix Cores. arXiv:2609.14845 [cs.AR] https://arxiv.org/abs/2609.14845

[46] S. Kullback and R. A. Leibler. 1951. On Information and Sufficiency. *The Annals of Mathematical Statistics* 22, 1 (1951), 79 – 86. doi:10.1214/aoms/1177729694

[47] Nuno P. Lopes, David Menendez, Santosh Nagarakatte, and John Regehr. 2015. Provably correct peephole optimizations with alive. In *Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation, Portland, OR, USA, June 15-17, 2015*, David Grove and Stephen M. Blackburn (Eds.). ACM, 22–32. doi:10.1145/2737924.2737965

[48] Matthieu Martel. 2017. Floating-Point Format Inference in Mixed-Precision. In *NASA Formal Methods - 9th International Symposium, NFM 2017, Moffett Field, CA, USA, May 16-18, 2017, Proceedings (Lecture Notes in Computer Science, Vol. 10227)*, Clark W. Barrett, Misty D. Davies, and Temesghen Kahsai (Eds.). 230–246. doi:10.1007/978-3-319-57288-8_16

[49] Henry Massalin. 1987. Superoptimizer - A Look at the Smallest Program. In *Proceedings of the Second International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS II), Palo Alto, California, USA, October 5-8, 1987*, Randy H. Katz and Martin Freeman (Eds.). ACM Press, 122–126. doi:10.1145/36206.36194

[50] William M. McKeeman. 1998. Differential Testing for Software. *Digital Technical Journal* 10, 1 (1998), 100–107.

[51] David Menendez, Santosh Nagarakatte, and Aarti Gupta. 2016. Alive-FP: Automated Verification of Floating Point Based Peephole Optimizations in LLVM. In *Static Analysis - 23rd International Symposium, SAS 2016, Edinburgh, UK, September 8-10, 2016, Proceedings (Lecture Notes in Computer Science, Vol. 9837)*, Xavier Rival (Ed.). Springer, 317–337. doi:10.1007/978-3-662-53413-7_16

[52] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. 2017. Pointer Sentinel Mixture Models. In *International Conference on Learning Representations*. https://openreview.net/forum?id=Byj72udxe

[53] Antoine Miné. 2001. A New Numerical Abstract Domain Based on Difference-Bound Matrices. In *Programs as Data Objects, Second Symposium, PADO 2001, Aarhus, Denmark, May 21-23, 2001, Proceedings (Lecture Notes in Computer Science, Vol. 2053)*, Olivier Danvy and Andrzej Filinski (Eds.). Springer, 155–172. doi:10.1007/3-540-44978-7_10

[54] Antoine Miné. 2006. The octagon abstract domain. *High. Order Symb. Comput.* 19, 1 (2006), 31–100. doi:10.1007/S10990-006-8609-1

[55] Antoine Miné. 2012. Abstract Domains for Bit-Level Machine Integer and Floating-point Operations. In *ATx'12/WInG'12: Joint Proceedings of the Workshops on Automated Theory eXploration and on Invariant Generation, Manchester, UK, June 2012 (EPiC Series in Computing, Vol. 17)*, Jacques D. Fleuriot, Peter Höfner, Annabelle McIver, and Alan Smaill (Eds.). EasyChair, 55–70. doi:10.29007/B63G

[56] Badreddine Noune, Philip Jones, Daniel Justus, Dominic Masters, and Carlo Luschi. 2022. 8-bit Numerical Formats for Deep Neural Networks. arXiv:2206.02915 [cs.LG] https://arxiv.org/abs/2206.02915

[57] NVIDIA Corporation. 2020. *NVIDIA A100 Tensor Core GPU Architecture Whitepaper*. Technical Report. NVIDIA Corporation. https://images.nvidia.com/aem-dam/en-zz/Solutions/data-center/nvidia-ampere-architecture-whitepaper.pdf

[58] E. Theodore L. Omtzigt and James Quinlan. 2022. Universal: Reliable, Reproducible, and Energy-Efficient Numerics. In *Conference on Next Generation Arithmetic*. Springer, 100–116.

[59] Andrew Or, Apurva Jain, Daniel Vega-Myhre, Jesse Cai, Charles David Hernandez, Zhenrui Zheng, Driss Guessous, Vasiliy Kuznetsov, Christian Puhrsch, Mark Saroufim, Supriya Rao, Thien Tran, and Aleksandar Samardžić. 2025. TorchAO: PyTorch-Native Training-to-Serving Model Optimization. arXiv:2507.16099 [cs.LG] https://arxiv.org/abs/2507.16099

[60] Pavel Panchekha, Alex Sanchez-Stern, James R. Wilcox, and Zachary Tatlock. 2015. Automatically improving accuracy for floating point expressions. In *Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation* (Portland, OR, USA) *(PLDI '15)*. Association for Computing Machinery, New York, NY, USA, 1–11. doi:10.1145/2737924.2737959

[61] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. 2019. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In *Advances in Neural Information Processing Systems*, H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett (Eds.), Vol. 32. Curran Associates, Inc., Red Hook, NY, USA, 8024–8035. https://proceedings.neurips.cc/paper_files/paper/2019/file/bdbca288fee7f92f2bfa9f7012727740-Paper.pdf

[62] Python Software Foundation. 2026. `decimal` — *Decimal Fixed-Point and Floating-Point Arithmetic*. Retrieved October 31, 2026 from https://docs.python.org/3/library/decimal.html

[63] Qwen Team. 2026. Qwen3.5: Towards Native Multimodal Agents. https://qwen.ai/blog?id=qwen3.5

[64] Bita Darvish Rouhani, Nitin Garegrat, Tom Savell, Ankit More, Kyung-Nam Han, Ritchie Zhao, Mathew Hall, Jasmine Klar, Eric Chung, Yuan Yu, Michael Schulte, Ralph Wittig, Ian Bratt, Nigel Stephens, Jelena Milanovic, John Brothers, Pradeep Dubey, Marius Cornea, Alexander Heinecke, Andres Rodriguez, Martin Langhammer, Summer Deng, Maxim Naumov, Paulius Micikevicius, Michael Siu, and Colin Verrilli. 2023. *OCP Microscaling Formats (MX) Specification, Version 1.0*. Technical Report. Open Compute Project. https://www.opencompute.org/documents/ocp-microscaling-formats-mx-v1-0-spec-final-pdf

[65] Pierre Roux. 2014. Innocuous Double Rounding of Basic Arithmetic Operations. *Journal of Formalized Reasoning* 7, 1 (Jan. 2014), 131–142. doi:10.6092/issn.1972-5787/4359

[66] Cindy Rubio-González, Cuong Nguyen, Hong Diep Nguyen, James Demmel, William Kahan, Koushik Sen, David H. Bailey, Costin Iancu, and David Hough. 2013. Precimonious: tuning assistant for floating-point precision. In *Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis* (Denver, Colorado) *(SC '13)*. Association for Computing Machinery, New York, NY, USA, Article 27, 12 pages. doi:10.1145/2503210.2503296

[67] Radu Rugina and Martin C. Rinard. 2000. Symbolic bounds analysis of pointers, array indices, and accessed memory regions. In *Proceedings of the 2000 ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI), Vancouver, Britith Columbia, Canada, June 18-21, 2000*, Monica S. Lam (Ed.). ACM, 182–195. doi:10.1145/349299.349325

[68] Siegfried M. Rump. 2016. IEEE754 Precision-k base-β Arithmetic Inherited by Precision-m Base-β Arithmetic for k < m. *ACM Trans. Math. Softw.* 43, 3, Article 20 (Dec. 2016), 15 pages. doi:10.1145/2785965

[69] David M. Russinoff. 2019. Formal Verification of Floating-Point RTL with ACL2. In *Workshop on Instruction Set Architecture Specification (SPISA)*. https://www.cl.cam.ac.uk/~jrh13/spisa19/paper_06.pdf

[70] Brett Saiki, Jackson Brough, Jonas Regehr, Jesus Ponce, Varun Pradeep, Aditya Akhileshwaran, Zachary Tatlock, and Pavel Panchekha. 2025. Target-Aware Implementation of Real Expressions. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1* (Rotterdam, Netherlands) *(ASPLOS '25)*. Association for Computing Machinery, New York, NY, USA, 1069–1083. doi:10.1145/3669940.3707277

[71] Brett Saiki, Bill Zorn, Cynthia Richey, and Zachary Tatlock. 2026. When Double Rounding is Correct. arXiv:2610.09005 [cs.PL] https://arxiv.org/abs/2610.09005

[72] Eric Schkufza, Rahul Sharma, and Alex Aiken. 2013. Stochastic superoptimization. In *Architectural Support for Programming Languages and Operating Systems, ASPLOS 2013, Houston, TX, USA, March 16-20, 2013*, Vivek Sarkar and Rastislav Bodík (Eds.). ACM, 305–316. doi:10.1145/2451116.2451150

[73] Eric Schkufza, Rahul Sharma, and Alex Aiken. 2014. Stochastic optimization of floating-point programs with tunable precision. In *ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI '14, Edinburgh, United Kingdom - June 09 - 11, 2014*, Michael F. P. O'Boyle and Keshav Pingali (Eds.). ACM, 53–64. doi:10.1145/2594291.2594302

[74] Mark Stephenson, Jonathan Babb, and Saman Amarasinghe. 2000. Bitwidth Analysis with Application to Silicon Compilation. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. 108–120. doi:10.1145/349299.349317

[75] Lintang Sutawika, Leo Gao, Hailey Schoelkopf, Stella Biderman, Jonathan Tow, Baber Abbasi, ben fattori, Charles Lovering, farzanehnakhaee70, Jason Phang, Anish Thite, Fazz, Aflah, Niklas Muennighoff, Thomas Wang, sdtblck, nopperl, gakada, tttyuntian, researcher2, Chris, Julen Etxaniz, Zdeněk Kasner, Khalid, Jeffrey Hsu, AndyZwei, Pawan Sasanka Ammanamanchi, Dirk Groeneveld, Ethan Smith, and Eric Tang. 2023. *EleutherAI/lm-evaluation-harness: Major refactor.* doi:10.5281/zenodo.10256836

[76] Philippe Tillet, H. T. Kung, and David Cox. 2019. Triton: an intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages* (Phoenix, AZ, USA) *(MAPL 2019)*. Association for Computing Machinery, New York, NY, USA, 10–19. doi:10.1145/3315508.3329973

[77] Benjamin Valpey, Xinyi Li, Sreepathi Pai, and Ganesh Gopalakrishnan. 2025. An SMT Formalization of Mixed-Precision Matrix Multiplication: Modeling Three Generations of Tensor Cores. In *NASA Formal Methods: 17th International Symposium, NFM 2025, Williamsburg, VA, USA, June 11–13, 2025, Proceedings*. Springer-Verlag, Berlin, Heidelberg, 360–379. doi:10.1007/978-3-031-93706-4_21

[78] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022*, Sanmi Koyejo, S. Mohamed, A. Agarwal, Danielle Belgrave, K. Cho, and A. Oh (Eds.). http://papers.nips.cc/paper_files/paper/2022/hash/9d5609613524ecf4f15af0f7b31abca4-Abstract-Conference.html

[79] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander M. Rush. 2020. HuggingFace's Transformers: State-of-the-art Natural Language Processing. arXiv:1910.03771 [cs.CL] https://arxiv.org/abs/1910.03771

[80] Peichen Xie, Shuotao Xu, Yang Wang, Fan Yang, and Mao Yang. 2026. Bit-Accurate Modeling of GPU Matrix Multiply-Accumulate Units: Demystifying Numerical Discrepancy and Accuracy. arXiv:2511.10909 [cs.AR] https://arxiv.org/abs/2511.10909

[81] David Kai Zhang and Alex Aiken. 2025. Automatic Verification of Floating-Point Accumulation Networks. In *Computer Aided Verification - 37th International Conference, CAV 2025, Zagreb, Croatia, July 23-25, 2025, Proceedings, Part III (Lecture Notes in Computer Science, Vol. 15933)*, Ruzica Piskac and Zvonimir Rakamaric (Eds.). Springer, 215–237. doi:10.1007/978-3-031-98682-6_12

[82] Bill Zorn. 2021. *Rounding*. Ph. D. Dissertation. University of Washington. http://hdl.handle.net/1773/48230

[83] Niko Zurstraßen, Nils Bosbach, and Rainer Leupers. 2025. FloppyFloat: An Open Source Floating Point Library for Instruction Set Simulators. In *2025 Design, Automation & Test in Europe Conference (DATE)*. 1–6. doi:10.23919/DATE64628.2025.10992803

A Full Semantics of FPy

Figures 17 and 18 give the full semantics of FPy for the syntax in Figure 3. Section 3.1 explains the rules specific to FPy (E-Arith, E-Ctx, E-Context, E-App); the rest are standard. Values and $\mathbb{R}$ evaluate to themselves, a variable to its value in the store, and indexing is zero-based. An exact operation applies its operator without rounding (E-Exact). A statement completes normally with an updated store or returns a value; a return ends a sequence (E-Seq-Return), and a loop unrolls one iteration at a time. A program runs its entry function under $\mathbb{R}$ and must return (E-Program). A program has no result if it indexes out of bounds, fails an assertion, builds a context from a record that ρ does not map, or reaches the end of a function without returning.

$$\boxed{\langle \sigma, C, e \rangle \Downarrow v}$$

$$\text{E-Const}\ \frac{}{\langle \sigma, C, n \rangle \Downarrow n} \qquad \text{E-Bool}\ \frac{v \in \{\text{true}, \text{false}\}}{\langle \sigma, C, v \rangle \Downarrow v} \qquad \text{E-Var}\ \frac{x \in \text{dom}(\sigma)}{\langle \sigma, C, x \rangle \Downarrow \sigma(x)} \qquad \text{E-List}\ \frac{\langle \sigma, C, e_i \rangle \Downarrow v_i \quad (1 \le i \le m)}{\langle \sigma, C, [\, e_1, \ldots, e_m \,] \rangle \Downarrow [\, v_1, \ldots, v_m \,]}$$

$$\text{E-Index}\ \frac{\langle \sigma, C, e_1 \rangle \Downarrow [\, v_1, \ldots, v_m \,] \qquad \langle \sigma, C, e_2 \rangle \Downarrow n \qquad n \in \{0, \ldots, m-1\}}{\langle \sigma, C, e_1[e_2] \rangle \Downarrow v_{n+1}}$$

$$\text{E-Record}\ \frac{\langle \sigma, C, e_i \rangle \Downarrow v_i \quad (1 \le i \le m)}{\langle \sigma, C, \{\, l_1 = e_1, \ldots, l_m = e_m \,\} \rangle \Downarrow \{\, l_1 = v_1, \ldots, l_m = v_m \,\}} \qquad \text{E-Proj}\ \frac{\langle \sigma, C, e \rangle \Downarrow \{\, l_1 = v_1, \ldots, l_m = v_m \,\} \qquad l = l_i}{\langle \sigma, C, e.l \rangle \Downarrow v_i}$$

$$\text{E-Real}\ \frac{}{\langle \sigma, C, \mathbb{R} \rangle \Downarrow \mathbb{R}} \qquad \text{E-Ctx}\ \frac{\langle \sigma, C, e \rangle \Downarrow v \qquad v \in \text{dom}(\rho)}{\langle \sigma, C, \text{ctx}\ e \rangle \Downarrow \text{ctx}\ v} \qquad \text{E-Arith}\ \frac{op \in \mathit{Arith} \qquad \langle \sigma, C, e_i \rangle \Downarrow v_i \quad (1 \le i \le k)}{\langle \sigma, C, op(e_1, \ldots, e_k) \rangle \Downarrow \text{rnd}_C([\![op(v_1, \ldots, v_k)]\!])}$$

$$\text{E-Exact}\ \frac{op \in \mathit{Exact} \qquad \langle \sigma, C, e_i \rangle \Downarrow v_i \quad (1 \le i \le k)}{\langle \sigma, C, op(e_1, \ldots, e_k) \rangle \Downarrow op(v_1, \ldots, v_k)} \qquad \text{E-App}\ \frac{\Phi(f) = (y, s) \qquad \langle \sigma, C, e \rangle \Downarrow v \qquad \langle [y \mapsto v], C, s \rangle \Downarrow_S \text{return}\ v'}{\langle \sigma, C, f(e) \rangle \Downarrow v'}$$

Fig. 17. Semantics of FPy: expressions.

For brevity, FPy includes list comprehensions, `zip`, `join`, and `sum` (Section 3.2); Figure 19 defines them. `zip` and `join` are exact operators (E-Exact).

$$\boxed{\langle \sigma, C, s\rangle \Downarrow_S o} \qquad \boxed{(\Phi, f_{main})(v) \Downarrow v'}$$

$$\textsc{E-Assign}\ \frac{\langle \sigma, C, e\rangle \Downarrow v}{\langle \sigma, C, x = e\rangle \Downarrow_S \text{normal } \sigma[x \mapsto v]} \qquad \textsc{E-Skip}\ \frac{}{\langle \sigma, C, \text{skip}\rangle \Downarrow_S \text{normal } \sigma} \qquad \textsc{E-Ret}\ \frac{\langle \sigma, C, e\rangle \Downarrow v}{\langle \sigma, C, \text{ret } e\rangle \Downarrow_S \text{return } v}$$

$$\textsc{E-Assert}\ \frac{\langle \sigma, C, e\rangle \Downarrow \text{true}}{\langle \sigma, C, \text{assert } e\rangle \Downarrow_S \text{normal } \sigma} \qquad \textsc{E-Seq-Normal}\ \frac{\langle \sigma, C, s_1\rangle \Downarrow_S \text{normal } \sigma' \qquad \langle \sigma', C, s_2\rangle \Downarrow_S o}{\langle \sigma, C, s_1\,; s_2\rangle \Downarrow_S o} \qquad \textsc{E-Seq-Return}\ \frac{\langle \sigma, C, s_1\rangle \Downarrow_S \text{return } v}{\langle \sigma, C, s_1\,; s_2\rangle \Downarrow_S \text{return } v}$$

$$\textsc{E-If-True}\ \frac{\langle \sigma, C, e\rangle \Downarrow \text{true} \qquad \langle \sigma, C, s_1\rangle \Downarrow_S o}{\langle \sigma, C, \text{if } e \text{ then } s_1 \text{ else } s_2\rangle \Downarrow_S o} \qquad \textsc{E-If-False}\ \frac{\langle \sigma, C, e\rangle \Downarrow \text{false} \qquad \langle \sigma, C, s_2\rangle \Downarrow_S o}{\langle \sigma, C, \text{if } e \text{ then } s_1 \text{ else } s_2\rangle \Downarrow_S o} \qquad \textsc{E-While-False}\ \frac{\langle \sigma, C, e\rangle \Downarrow \text{false}}{\langle \sigma, C, \text{while } e \text{ do } s\rangle \Downarrow_S \text{normal } \sigma}$$

$$\textsc{E-While-True}\ \frac{\langle \sigma, C, e\rangle \Downarrow \text{true} \qquad \langle \sigma, C, s\,; \text{while } e \text{ do } s\rangle \Downarrow_S o}{\langle \sigma, C, \text{while } e \text{ do } s\rangle \Downarrow_S o} \qquad \textsc{E-Context}\ \frac{\langle \sigma, \mathbb{R}, e\rangle \Downarrow C' \qquad \langle \sigma, C', s\rangle \Downarrow_S o}{\langle \sigma, C, \text{with } e \text{ in } s\rangle \Downarrow_S o}$$

$$\textsc{E-Program}\ \frac{\Phi(f_{main}) = (y, s) \qquad \langle [y \mapsto v], \mathbb{R}, s\rangle \Downarrow_S \text{return } v'}{(\Phi, f_{main})(v) \Downarrow v'}$$

Fig. 18. Semantics of FPy: statements and programs.

```
1  def sum(xs) =
2    if len(xs) == 0 then
3      ret 0
4    else
5      s = xs[0];
6      i = 1;
7      while i < len(xs) do
8        s = s + xs[i];
9        with ℝ in i = i + 1;
10     ret s
```

$$\text{join}([v_1, \ldots, v_m], [w_1, \ldots, w_n]) = [v_1, \ldots, v_m, w_1, \ldots, w_n]$$
$$\text{zip}([v_1, \ldots, v_m], [w_1, \ldots, w_n]) = [[v_1, w_1], \ldots, [v_k, w_k]] \quad \text{where } k = \min(m, n)$$

$$\textsc{E-Comp}\ \frac{\langle \sigma, C, e_2\rangle \Downarrow [v_1, \ldots, v_m] \qquad \langle \sigma[x \mapsto v_i], C, e_1\rangle \Downarrow u_i\ (1 \le i \le m)}{\langle \sigma, C, [\, e_1 \text{ for } x \text{ in } e_2\,]\rangle \Downarrow [u_1, \ldots, u_m]}$$

Fig. 19. Definitions of `sum` (left), and of `join`, `zip`, and list comprehensions (right). The function `sum` adds a list's elements in order, each addition rounding under the caller's context. A comprehension over pairs, as in `for xi, yi in zip(x, y)`, binds each variable to its component.

B Format Inference

This appendix completes format inference (Section 4.2). Figure 20 recalls abstract formats from prior work [71]. Table 4 gives our abstract operators, which refine its arithmetic in two ways: each precision is clamped to what the format's bound allows ($\hat{p}$), and a product with a power of two keeps the other operand's precision (*mulprec*).

Each operator in Table 4 has a case for finite sets V_i and a case for instances $\mathcal{A}_i = \mathcal{A}(p_i, exp_i, b_i, S_i)$. A set meeting an instance is first replaced by $\lceil V \rceil$, the smallest instance containing it; if V holds a value not of the form $m \times 2^e$ for integers m and e, $\lceil V \rceil$ does not exist and the result is $\mathbb{R}^*$. The cases

on instances use three helpers:

$$bits(b,e) = \lceil \log_2(\lfloor b/2^e \rfloor + 1) \rceil$$
$$\hat{p} = \min(p,\ bits(b, exp))$$
$$mulprec(q_1,q_2) = \begin{cases} \max(q_1,q_2) & \text{if } 1 \in \{q_1,q_2\} \\ \max(1, q_1+q_2) & \text{otherwise} \end{cases}$$

Here, $bits(b,e)$ is the number of digits a magnitude up to b needs at quantum 2^e. For special values, $S_\oplus$ holds $S_1 \cup S_2$, plus NaN if one operand may be $+\infty$ and the other $-\infty$; $S_\otimes$ holds $S_1 \cup S_2$, plus $\pm\infty$ and NaN if either operand may be infinite ($\infty \times 0$ is NaN); $-S$ negates the infinities in S; and $|S|$ replaces $-\infty$ by $+\infty$.

$$\begin{array}{rcl} \mathcal{A}(p, exp, b, S) & = & \{\, m \times 2^e \mid m, e \in \mathbb{Z},\ |m| < 2^p,\ e \geq exp,\ |m \times 2^e| \leq b \,\} \cup S \\ F_1 \subseteq F_2 & \iff & exp_1 \geq exp_2 \wedge b_1 \leq b_2 \wedge S_1 \subseteq S_2 \wedge (p_1 \leq p_2 \vee (exp_1 > -\infty \wedge b_1 \leq 2^{exp_1+p_2})) \\ F_1 \oplus F_2 & = & \mathcal{A}(\lceil \log_2(\lfloor (b_1+b_2)/2^{\min(exp_1,exp_2)} \rfloor + 1) \rceil,\ \min(exp_1,exp_2),\ b_1+b_2,\ S_\oplus) \\ F_1 \otimes F_2 & = & \mathcal{A}(p_1+p_2,\ exp_1+exp_2,\ b_1 b_2,\ S_\otimes) \end{array}$$

Fig. 20. Abstract formats: representable values, containment, and arithmetic. Here, F_i is $\mathcal{A}(p_i, exp_i, b_i, S_i)$.

Abstract values extend those of Figure 6 with lists, records, a value $\bullet$ for booleans, and a value $\top$ for any value:

$$a ::= F \mid \kappa \mid [\ a\ ] \mid \{\, l_1 = a_1, \ldots, l_m = a_m \,\} \mid \bullet \mid \top$$

A list is bounded by one abstract value for all of its elements, and a record field by field; joins and the order $\sqsubseteq$ follow this structure, and abstract values of different kinds join to $\top$. Format environments Γ map variables to abstract values. Operators are lifted to abstract values, giving $\bullet$ for comparisons and other boolean-valued operators.

Table 5 gives all cases of format inference, extending Figure 7. A conditional refines the environment by its condition in each branch (C$^\sharp$, Section 4.2) and joins the results; a loop analyzes its body under an invariant Γ^* that contains the environment on entry and after each iteration; and a context statement analyzes its body under the abstract context of its expression, evaluated under $\mathbb{R}$. In the table, known(a) is ctx r if a describes exactly one record r and ctx r induces a format, and ? otherwise; and the sum of a list bounded by $[\ F\ ]$ with L elements is bounded by F_L, where $F_0 = \{0\}$, $F_1 = F$, and $F_{i+1} = \mathrm{rnd}^\sharp_\kappa(F_i +^\sharp F)$. A program (Φ, f_{main}) with $\Phi(f_{main}) = (y, s)$, run on an argument bounded by a, returns a value bounded by a', where $\langle \Gamma', a' \rangle = [\![s]\!]^\sharp_{\mathbb{R}}\ [y \mapsto a]$.

Table 4. Abstract operators $op^\sharp$ on finite sets V_i and on instances $\mathcal{A}_i = \mathcal{A}(p_i, exp_i, b_i, S_i)$.

Operator	On finite sets V	On instances $\mathcal{A}(p, exp, b, S)$
$x + y$	$\{\, a + b \mid a \in V_1,\ b \in V_2 \,\}$	$\mathcal{A}(\max(1, bits(b_1 + b_2, e)),\ e,\ b_1 + b_2,\ S_\oplus)$, $e = \min(exp_1, exp_2)$
$x - y$	$\{\, a - b \mid a \in V_1,\ b \in V_2 \,\}$	same as $x + (-y)$
$x \times y$	$\{\, a \cdot b \mid a \in V_1,\ b \in V_2 \,\}$	$\mathcal{A}(mulprec(\hat{p}_1, \hat{p}_2),\ exp_1 + exp_2,\ b_1 b_2,\ S_\otimes)$
$-x$	$\{\, -a \mid a \in V \,\}$	$\mathcal{A}(p, exp, b, -S)$
$\lvert x \rvert$	$\{\, \lvert a \rvert \mid a \in V \,\}$	$\mathcal{A}(p, exp, b, \lvert S \rvert)$
$\mathrm{logb}(x)$	$\mathrm{logb}^\sharp(\lceil V \rceil)$	$\mathcal{A}(\max(1, bits(b', 0)),\ 0,\ b',\ S')$, where $b' = \max(\lvert exp \rvert,\ \lvert \lfloor \log_2 b \rfloor \rvert)$ and S' holds $-\infty$ (for $\mathrm{logb}(0)$), $+\infty$ if S has an infinity, and NaN if S does, if $exp > -\infty$ and $0 < b < \infty$; otherwise $\mathbb{R}^*$
2^x	$\exp2^\sharp(\lceil V \rceil)$	$\mathcal{A}(1,\ -\lfloor b \rfloor,\ 2^{\lfloor b \rfloor},\ S')$, where S' holds $+\infty$ and NaN if S does, if $exp \geq 0$ and $b < \infty$; otherwise $\mathbb{R}^*$
$\min(x, y)$, $\max(x, y)$	$\{\, \min(a, b) \mid a \in V_1,\ b \in V_2 \,\}$, dually for max	$\mathcal{A}_1 \sqcup \mathcal{A}_2$
$\mathrm{round}(x)$, $\mathrm{cast}(x)$	V	$\mathcal{A}(p, exp, b, S)$
all others	$\mathbb{R}^*$	$\mathbb{R}^*$

Table 5. All cases of format inference, extending Figure 7. Pairs join componentwise and environments pointwise, Γ^* is a loop invariant, and $\{\}$ marks no returned value.

Expression e	$[\![e]\!]^\sharp_\kappa\, \Gamma$
n	$\{n\}$
true, false	$\bullet$
x	$\Gamma(x)$
$[e_1, \ldots, e_m]$	$[\ [\![e_1]\!]^\sharp_\kappa\, \Gamma \sqcup \cdots \sqcup [\![e_m]\!]^\sharp_\kappa\, \Gamma\ ]$
$e_1[e_2]$	a, where $[\ a\ ] = [\![e_1]\!]^\sharp_\kappa\, \Gamma$
$\{\, l_1 = e_1, \ldots, l_m = e_m \,\}$	$\{\, l_1 = [\![e_1]\!]^\sharp_\kappa\, \Gamma, \ldots, l_m = [\![e_m]\!]^\sharp_\kappa\, \Gamma \,\}$
$e.l$	a_i, where $\{\ldots, l = a_i, \ldots\} = [\![e]\!]^\sharp_\kappa\, \Gamma$
$\mathbb{R}$	$\mathbb{R}$
ctx e	$\mathrm{known}([\![e]\!]^\sharp_\kappa\, \Gamma)$
$op(e_1, \ldots, e_k)$, $op \in Arith$	$\mathrm{rnd}^\sharp_\kappa(op^\sharp([\![e_1]\!]^\sharp_\kappa\, \Gamma, \ldots, [\![e_k]\!]^\sharp_\kappa\, \Gamma))$
$op(e_1, \ldots, e_k)$, $op \in Exact$	$op^\sharp([\![e_1]\!]^\sharp_\kappa\, \Gamma, \ldots, [\![e_k]\!]^\sharp_\kappa\, \Gamma)$
$\mathrm{sum}(e_1)$	F_L, where $L = \mathrm{len}(e_1)$ and $[\ F\ ] = [\![e_1]\!]^\sharp_\kappa\, \Gamma$
$f(e_1)$	a, where $\Phi(f) = (y, s)$ and $\langle \Gamma', a \rangle = [\![s]\!]^\sharp_\kappa\, [\, y \mapsto [\![e_1]\!]^\sharp_\kappa\, \Gamma\,]$
Statement s	**$[\![s]\!]^\sharp_\kappa\, \Gamma$**
$x = e$	$\langle \Gamma[x \mapsto [\![e]\!]^\sharp_\kappa\, \Gamma],\ \{\}\rangle$
skip	$\langle \Gamma, \{\}\rangle$
ret e	$\langle \Gamma, [\![e]\!]^\sharp_\kappa\, \Gamma\rangle$
assert e	$\langle \mathrm{C}^\sharp[\![e]\!]_\kappa\, \Gamma, \{\}\rangle$
$s_1\,;s_2$	$\langle \Gamma_2, a_1 \sqcup a_2\rangle$, where $\langle \Gamma_1, a_1\rangle = [\![s_1]\!]^\sharp_\kappa\, \Gamma$ and $\langle \Gamma_2, a_2\rangle = [\![s_2]\!]^\sharp_\kappa\, \Gamma_1$
if e then s_1 else s_2	$[\![s_1]\!]^\sharp_\kappa(\mathrm{C}^\sharp[\![e]\!]_\kappa\, \Gamma) \sqcup [\![s_2]\!]^\sharp_\kappa(\mathrm{C}^\sharp[\![\neg e]\!]_\kappa\, \Gamma)$
while e do s_1	$\langle \mathrm{C}^\sharp[\![\neg e]\!]_\kappa\, \Gamma^*,\ a\rangle$, where $\langle \Gamma', a\rangle = [\![s_1]\!]^\sharp_\kappa(\mathrm{C}^\sharp[\![e]\!]_\kappa\, \Gamma^*)$ and $\Gamma \sqcup \Gamma' \sqsubseteq \Gamma^*$
with e in s_1	$[\![s_1]\!]^\sharp_{\kappa'}\, \Gamma$, where $\kappa' = [\![e]\!]^\sharp_{\mathbb{R}}\, \Gamma$

C Digit-Bound Analysis

This appendix completes digit-bound analysis (Section 4.3). Table 6 gives the abstract operators op^{D} and roundings $\mathrm{rnd}_\kappa^{\mathsf{D}}$ on digit intervals, extending those of Figure 9. Operands have intervals $[l_i, m_i]$, and an integer operator's interval is that of the corresponding real operator. In the rounding rows, e is the rounded expression, and 2^q is the smallest subnormal; each row is also joined with the intervals of the finite values that the context gives for special values and overflowing results, such as $\pm b$ under RTZ. On the empty interval, $\pm^{\mathsf{D}}$ returns the other operand, and $\times^{\mathsf{D}}$ and $\mathrm{rnd}_\kappa^{\mathsf{D}}$ return it.

Like format inference, the analysis propagates an environment Δ through statements. Unlike format inference, it makes a single pass and generates constraints Ψ instead of computing bounds; a solver optimizes them afterward (Section 4.3). Δ maps each variable to a description: for a number, a digit interval $[l, m]$ of terms and possibly a value term; for a list, one description for all of its elements; for a record, one description per field; and for a context, an abstract context. An abstract context is a context as in format inference whose integer parameters may be terms, such as $\mathtt{Fixed}(\mathit{val}(\mathtt{emax}) - \mathsf{k}, \mathtt{RTZ})$, or ? if it is unknown; the abstract context of $\mathtt{ctx}\ e$ takes $\mathit{val}(e.l)$ as each parameter l, or is ? if some field has no value. We write $d(e)$ for the description of an expression e. Zero has no digits, so its *lsb* and *msb* are $+\infty$ and $-\infty$. One set of variables describes all elements of a list: each element satisfies Ψ under its own assignment, which agrees with the others on all variables but the elements'; in the case for sum, *lo* and *hi* therefore mention no variable of the elements.

Table 7 gives all cases of digit-bound analysis, extending Figure 9. A statement s maps an environment Δ to $[\![s]\!]_\kappa^{\mathsf{D}}\,\Delta = \langle \Delta', d\rangle$, where d describes the value s returns. Where control flow joins, the join $\Delta_1 \sqcup \Delta_2$ gives each variable fresh terms: an interval within $[\min(l_1, l_2), \max(m_1, m_2)]$ and a value between the two values, or no value if either side has none. A loop analyzes its body once, from an invariant Δ^I that gives each variable the body mutates fresh terms bounded only by the digits of its format. A context statement analyzes its body under the abstract context of its expression, and a call analyzes the body of f again, with only its argument. For precision, the analysis also uses inferred formats: it constrains each expression e by $\mathit{digits}(e) \subseteq \mathit{digits}(\mathrm{fmt}(e))$ (Section 4.3), and if $\mathrm{fmt}(e) \subseteq \mathbb{Z} \cap [a, b]$, it gives e a value with $a \le \mathit{val}(e) \le b$.

Table 6. Abstract operators op^{D} and roundings $\mathrm{rnd}_\kappa^{\mathsf{D}}$ on digit intervals, where $p = \mathsf{nb} - \mathsf{es}$.

Operator	Digit interval	Context κ	$\mathrm{rnd}_\kappa^{\mathsf{D}}([l, m])$
$x \pm y$	$[\min(l_1, l_2),\ \max(m_1, m_2) + 1]$	$\mathbb{R}$	$[l, m]$
$x \times y$	$[l_1 + l_2,\ m_1 + m_2 + 1]$	$\mathtt{Fixed}(\tau, \mathtt{RTZ})$	$[\max(l, \tau), m]$
$-x$, $\lvert x\rvert$, $\mathrm{round}(x)$	$[l, m]$	$\mathtt{Fixed}(\tau, rm)$	$[\max(l, \tau), \max(m + 1, \tau)]$
$\min(x, y)$, $\max(x, y)$	$[\min(l_1, l_2),\ \max(m_1, m_2)]$	$\mathtt{IEEE754}(\mathsf{es}, \mathsf{nb}, \mathtt{RTZ})$	$[\max(l, m - p + 1),\ m]$
2^x	$[\mathit{val}(x), \mathit{val}(x)]$	$\mathtt{IEEE754}(\mathsf{es}, \mathsf{nb}, rm)$	$[\max(l, \mathit{msb}(e) - p + 1),\ \max(m + 1, q)]$
all others	$[-\infty, +\infty]$	? and all others	$[l, +\infty]$

Table 7. All cases of digit-bound analysis, extending Figure 9. Pairs join componentwise, and $\{\}$ marks no returned value.

Expression e	Context	$val(e)$	$digits(e)$
x	κ	$d(x) = \Delta(x)$	
$n \neq \mathrm{NaN}$	κ	$= n$ if $n \in \mathbb{Z}$	$= [\inf D(n), \sup D(n)]$
$e_1 \pm e_2$	$\mathbb{R}$	$= val(e_1) \pm val(e_2)$	$\subseteq digits(e_1) \pm^{\mathrm{D}} digits(e_2)$
$-e_1$	$\mathbb{R}$	$= -val(e_1)$	$= digits(e_1)$
$\mathrm{logb}(e_1)$	$\mathbb{R}$	$= msb(e_1)$	$\subseteq [0, \infty]$
$\max(e_1, \ldots, e_k)$	$\mathbb{R}$	$\geq val(e_i)$	$\subseteq [\min_i lsb(e_i), \max_i msb(e_i)]$
$\min(e_1, \ldots, e_k)$	$\mathbb{R}$	$\leq val(e_i)$	$\subseteq [\min_i lsb(e_i), \max_i msb(e_i)]$
$\mathrm{sum}(e_1), \mathrm{len}(e_1) = L \geq 1$	$\mathbb{R}$		$\subseteq [lo, hi + \lceil \log_2 L \rceil]$ if $\Psi \models digits(e_1) \subseteq [lo, hi]$
$op(e_1, \ldots, e_k)$, $op \in Arith$	κ		$\subseteq \mathrm{rnd}^{\mathrm{D}}_{\kappa}(op^{\mathrm{D}}(digits(e_1), \ldots, digits(e_k)))$
$[e_1, \ldots, e_m]$	κ		$\subseteq [\min_i lsb(e_i), \max_i msb(e_i)]$
$e_1[e_2]$	κ	$= val(e_1)$	$= digits(e_1)$
$f(e_1)$	κ	$d(f(e_1)) = d$, where $\Phi(f) = (y, s)$ and $\langle \Delta', d \rangle = [\![s]\!]^{\mathrm{D}}_{\kappa}\, [y \mapsto d(e_1)]$	
Statement s	$[\![s]\!]^{\mathrm{D}}_{\kappa}\, \Delta$		
$x = e$	$\langle \Delta[x \mapsto d(e)], \{\} \rangle$		
skip, assert e	$\langle \Delta, \{\} \rangle$		
ret e	$\langle \Delta, d(e) \rangle$		
$s_1 \,;\, s_2$	$\langle \Delta_2, d_1 \sqcup d_2 \rangle$, where $\langle \Delta_1, d_1 \rangle = [\![s_1]\!]^{\mathrm{D}}_{\kappa}\, \Delta$ and $\langle \Delta_2, d_2 \rangle = [\![s_2]\!]^{\mathrm{D}}_{\kappa}\, \Delta_1$		
if e then s_1 else s_2	$[\![s_1]\!]^{\mathrm{D}}_{\kappa}\, \Delta \sqcup [\![s_2]\!]^{\mathrm{D}}_{\kappa}\, \Delta$		
while e do s_1	$\langle \Delta^I, d \rangle$, where $\langle \Delta', d \rangle = [\![s_1]\!]^{\mathrm{D}}_{\kappa}\, \Delta^I$		
with e in s_1	$[\![s_1]\!]^{\mathrm{D}}_{\kappa'}\, \Delta$, where κ' is the abstract context of e		

D Mechanization

Our Lean mechanization is included as supplementary material. Table 8 maps the main results of Sections 4 and 5 to their Lean theorems (in namespace `Fpy`); each rewrite is proved on its own.

Table 8. Lean theorems for the main results of Sections 4 and 5.

Result	Lean theorem
Theorem 4.1 (expressions, statements)	`eval_sound`, `exec_sound`
Our formats meet the interface of Section 4.2	`fpyAnalysis_sound`
Theorem 4.2	`DigitBound.eval_format`, `DigitBound.run_format`
Executions satisfy Ψ (proof of Theorem 4.2)	`DigitBound.eval_solution`, `DigitBound.exec_sound`
Elim, Insert	`equiv_elim`, `equiv_insert`
Split	`equiv_split`
Special	`equiv_special`
Overflow (and by a threshold)	`equiv_overflow`, `equiv_overflow_threshold`
FloatToFixed, Rescale	`equiv_floatToFixed`, `equiv_rescale`
Summation in format inference	`eval_sum`
Summation in digit-bound analysis	`DigitBound.eval_sum_sound`

E Evaluation

This appendix gives details of the evaluation (Section 7). Table 9 lists the matrix-multiply-accumulate units that MMA-Sim models [80], from which Section 7.1 take their designs. A row with several input types, such as FP8/6/4, stands for several designs, and rows with the same arithmetic share a design: every FP64 row is a chain of FP64 fused multiply-adds, and the FP32 rows of CDNA1–3 a chain of FP32 ones; these are the two FMA chains of Section 7.1. FPy compiles every design except CDNA1's BF16 and FP16 instructions, which add their products and accumulator exactly before rounding, so their sums need 280 to 525 bits.

For the GPU comparison (Section 7.1.2), we optimize MMA-Sim's GPU code in two ways that leave its results bit for bit the same. First, MMA-Sim branches on values computed on the GPU and updates tensors through boolean masks, so the CPU waits for the GPU many times per instruction; we replace these with `torch.where`, so it never waits. Second, in decode, where each layer's shapes repeat every step, we capture each layer's call once as a CUDA graph and replay it, as LLM serving systems do, wherever the graph fits in memory. Optimized, MMA-Sim is 1.47× faster per design (median); against MMA-Sim as released, FPy's geometric-mean speedup is 162×.

Table 9. Matrix-multiply-accumulate units modeled by MMA-Sim. L is the length of the input vectors. LoC is the number of lines in the FPy specification. ✓: FPy compiles a design for the row; ✗: FPy cannot compile the specification.

Arch.	Input	Output	L	LoC	FPy
Volta	FP16	FP32	4	48	✓
	FP16	FP16	4	48	✓
Turing	FP16	FP32	8	48	✓
	FP16	FP16	8	48	✓
Ampere	FP64	FP64	1	6	✓
	TF32	FP32	4	48	✓
	BF16	FP32	8	48	✓
	FP16	FP32	8	48	✓
	FP16	FP16	8	48	✓
Ada Lovelace	FP64	FP64	1	6	✓
	TF32	FP32	4	48	✓
	BF16/FP16	FP32	8	48	✓
	FP16	FP16	8	48	✓
	FP8	FP32	16	48	✓
	FP8	FP16	16	48	✓
Hopper	FP64	FP64	1	6	✓
	TF32	FP32	8	48	✓
	BF16/FP16	FP32	16	48	✓
	FP16	FP16	16	48	✓
	FP8	FP32	32	48	✓
	FP8	FP16	32	48	✓
Blackwell	FP64	FP64	1	6	✓
	TF32	FP32	8	48	✓
	BF16/FP16	FP32	16	48	✓
	FP16	FP16	16	48	✓
	FP8/6/4	FP32	32	48	✓
	FP8/6/4	FP16	32	48	✓
	MXFP8/6/4	FP32	32	49	✓
	MXFP4/NVFP4	FP32	64	55	✓

Arch.	Input	Output	L	LoC	FPy
RTX Blackwell	FP64	FP64	1	6	✓
	TF32	FP32	8	48	✓
	BF16/FP16	FP32	16	48	✓
	FP16	FP16	16	48	✓
	FP8/6/4	FP32	32	48	✓
	FP8/6/4	FP16	32	48	✓
	MXFP8/6/4	FP32	32	49	✓
	MXFP4/NVFP4	FP32	64	55	✓
CDNA1	FP32	FP32	1	6	✓
	BF16	FP32	2	33	✗
	FP16	FP32	4	33	✗
CDNA2	FP64	FP64	1	6	✓
	FP32	FP32	1	6	✓
	BF16	FP32	2	25	✓
	FP16	FP32	4	25	✓
	BF16 (_1k)	FP32	4	25	✓
CDNA3	FP64	FP64	1	6	✓
	FP32	FP32	1	6	✓
	TF32	FP32	4	44	✓
	BF16/FP16	FP32	8	44	✓
	FP8	FP32	16	51	✓